

Distributed System Models and Enabling Technologies

Islamic University of Gaza
Faculty of Information Technology
Distributed and Cloud Systems (SICT 4401)
Dr. Eng. Rebhi Baraka (rbaraka@iugaza.edu.ps)

Outline

- Scalable Computing over the Internet
- Technologies for Network-Based Systems
- System Models for Distributed and Cloud Computing
- Software Environments for Distributed Systems and Clouds
- Performance, Security, and Energy Efficiency

Scalable Computing over the Internet

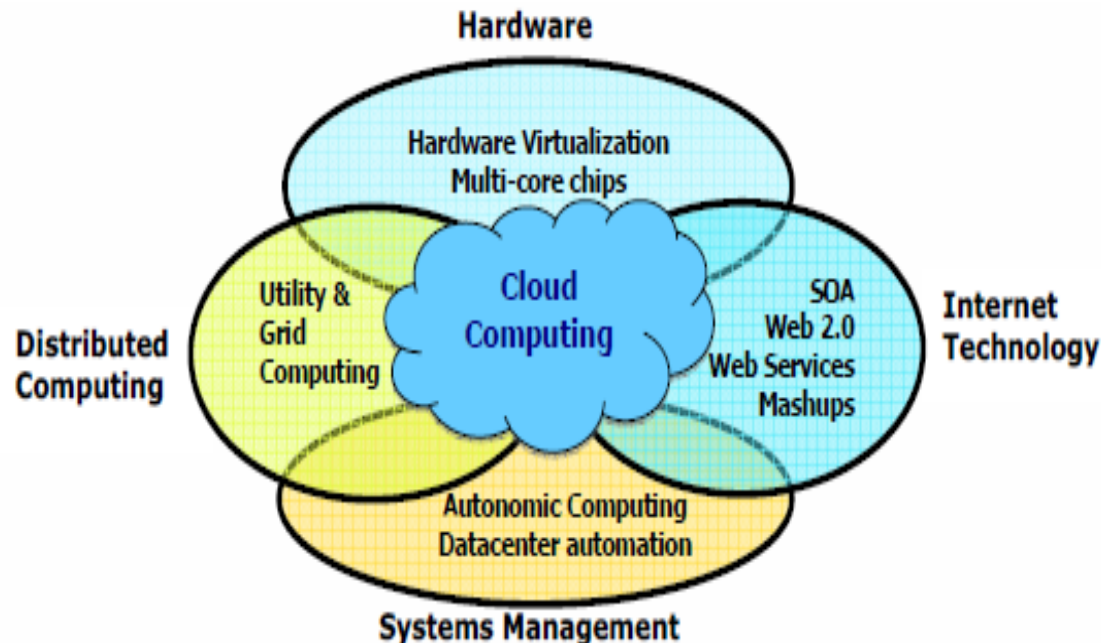
- Instead of using a centralized computer to solve computational problems, a parallel and distributed computing system uses multiple computers to solve large-scale problems over the Internet.
- Distributed computing becomes data-intensive and network-centric.
- We look into evolutionary changes in machine architecture, operating system platform, network connectivity, and application workload.

Convergence of Technologies

Data Deluge Enabling New Challenges

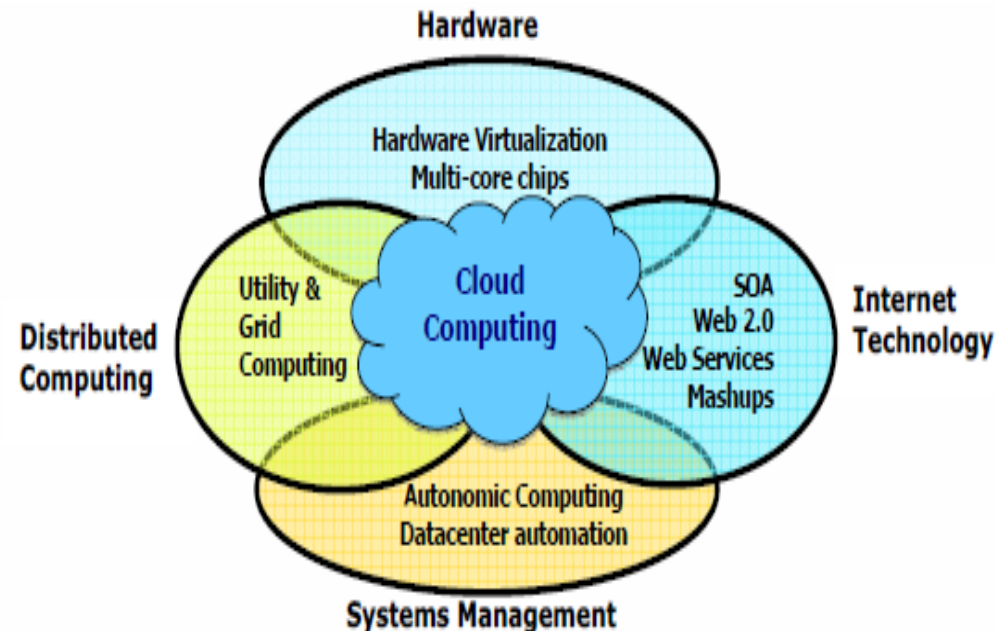
Cloud computing is enabled by the convergence of technologies in four areas:

- Hard- ware virtualization and multi-core chips,
- Utility and grid computing,
- SOA, Web 2 .0, and WS mashups, and
- Autonomic computing and data center automation.



Convergence of Technologies

- Hardware virtualization and multicore chips enable the existence of dynamic configurations in the cloud.
- Utility and grid computing technologies lay the necessary foundation for computing clouds.
- Recent advances in SOA, Web 2.0, and mashups of platforms are pushing the cloud another step forward.
- Finally, achievements in autonomic computing and automated data center operations contribute to the rise of cloud computing.

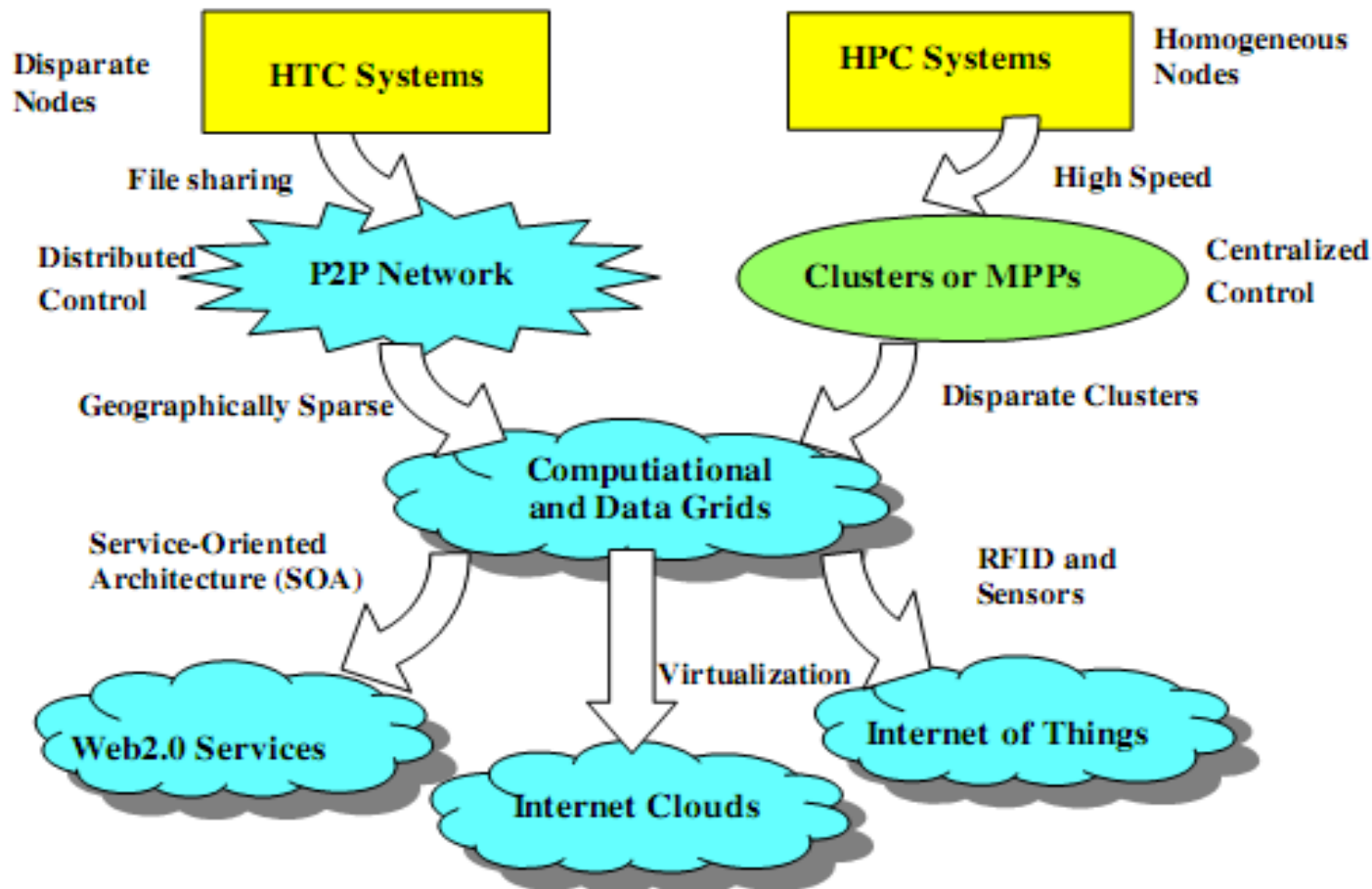


From Desktop/HPC/Grids to Internet Clouds in 30 Years

- HPC moving from centralized supercomputers to geographically distributed desktops, desksides, clusters, and grids to clouds over last 30 years
- R/D efforts on HPC, clusters, Grids, P2P, and virtual machines has laid the foundation of cloud computing that has been greatly advocated since 2007
- Location of computing infrastructure in areas with lower costs in hardware, software, datasets, space, and power requirements – moving from desktop computing to datacenter-based clouds

Clouds and Internet of Things

Evolutionary trend toward parallel, distributed, and cloud computing with clusters, MPPs, P2P networks, grids, clouds, web services, and the Internet of Things.



HPC: High-Performance Computing

HTC: High-Throughput Computing

P2P: Peer to Peer

MPP: Massively Parallel Processors

Computing Paradigm Distinctions

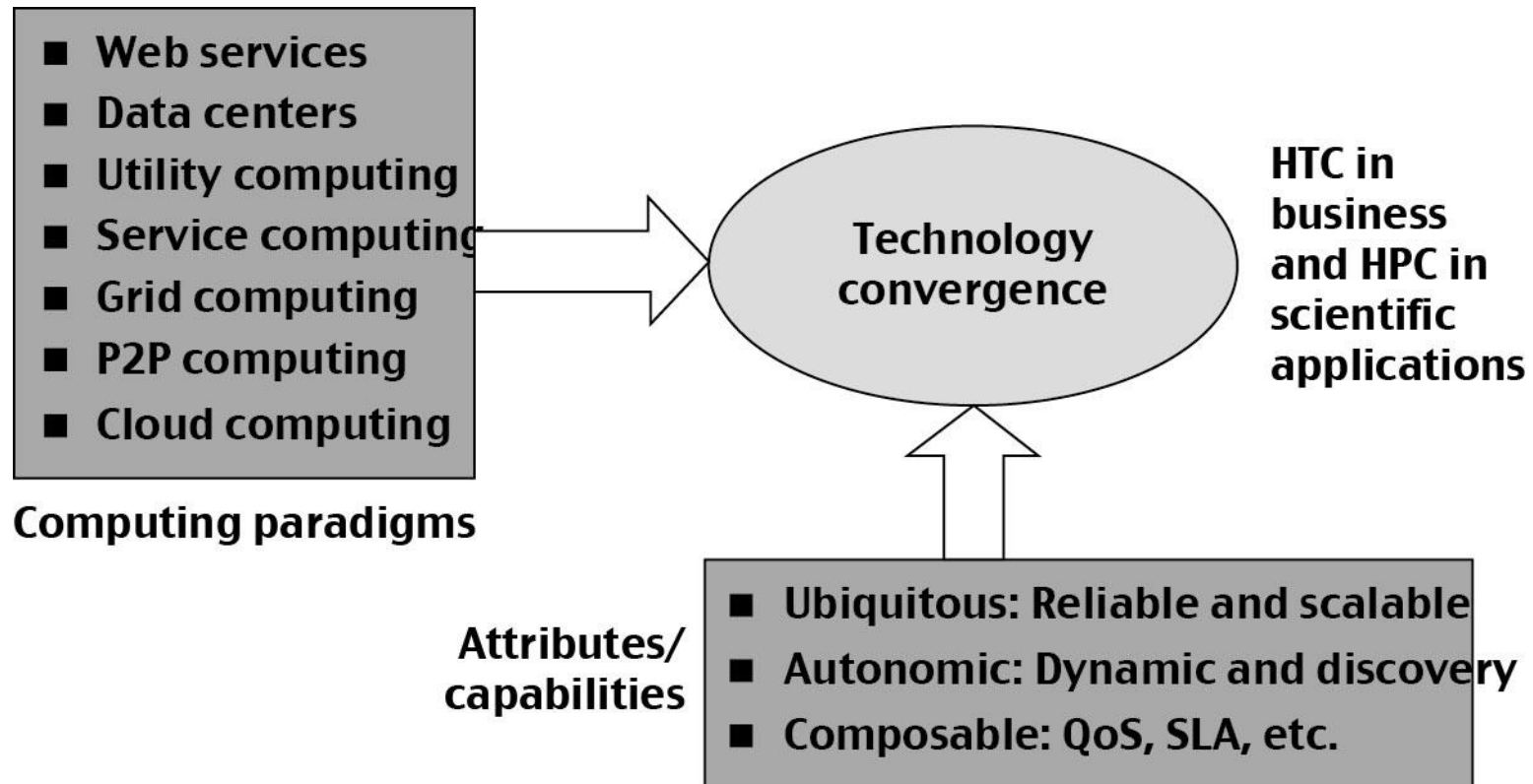
- Centralized Computing
 - All computer resources are centralized in one physical system.
- Parallel Computing
 - All processors are either tightly coupled with central shared memory or loosely coupled with distributed memory
- Distributed Computing
 - Field of CS/CE that studies distributed systems. A distributed system consists of multiple autonomous computers, each with its own private memory, communicating over a network.
- Cloud Computing
 - An Internet cloud of resources that may be either centralized or decentralized. The cloud applies to parallel or distributed computing or both. Clouds may be built from physical or virtualized resources.

HPC and HTC applications

- Key applications that have driven the development of parallel and distributed systems.
- All applications demand computing economics, web-scale data collection, system reliability, and scalable performance.

Domain	Specific Applications
Science and engineering	Scientific simulations, genomic analysis, etc. Earthquake prediction, global warming, weather forecasting, etc.
Business, education, services industry, and health care	Telecommunication, content delivery, e-commerce, etc. Banking, stock exchanges, transaction processing, etc. Air traffic control, electric power grids, distance education, etc. Health care, hospital automation, telemedicine, etc.
Internet and web services, and government applications	Internet search, data centers, decision-making systems, etc. Traffic monitoring, worm containment, cyber security, etc. Digital government, online tax return processing, social networking, etc.
Mission-critical applications	Military command and control, intelligent systems, crisis management, etc.

Technology Convergence toward HPC for Science and HTC for Business: *Utility Computing*



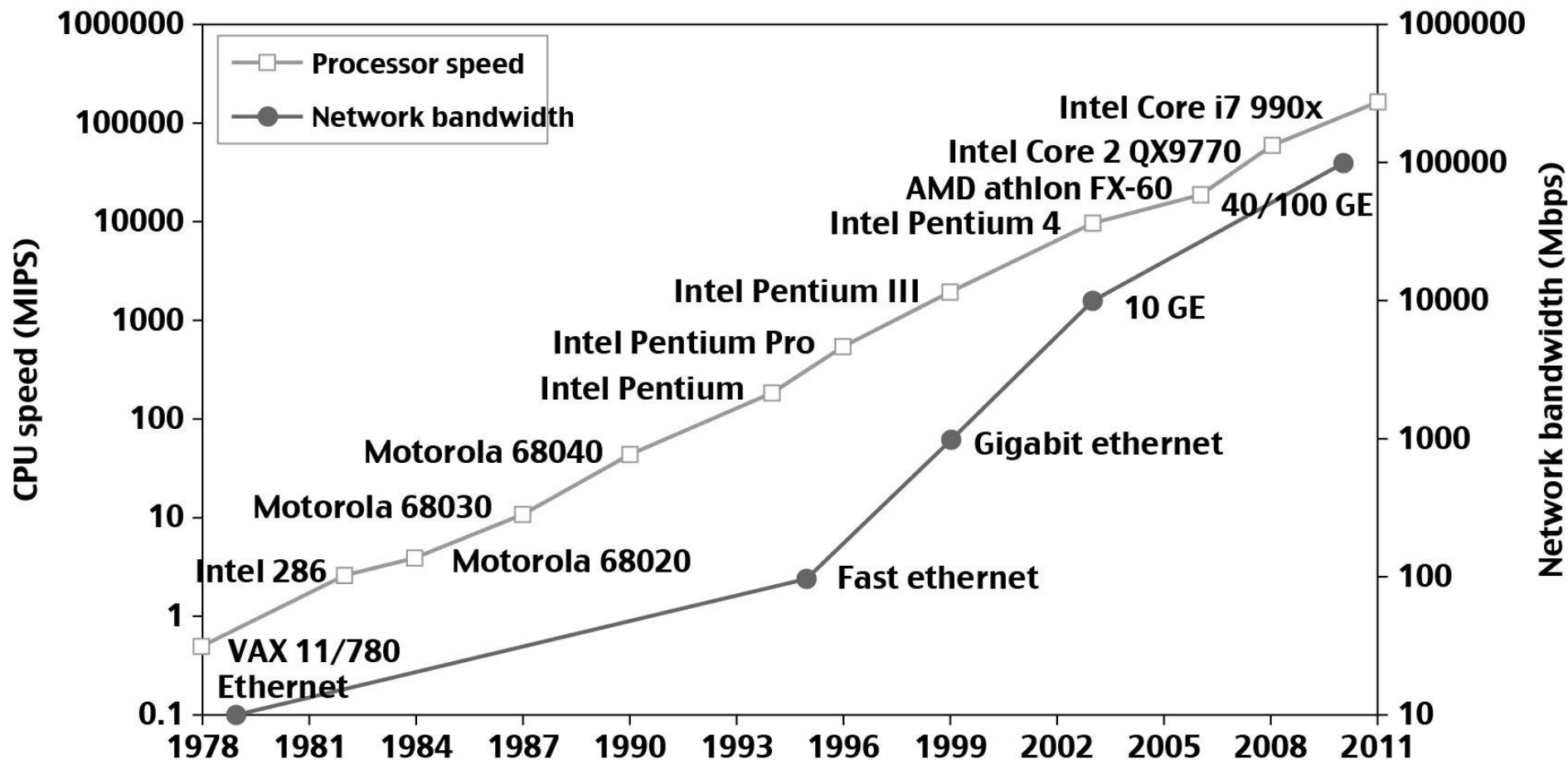
Technologies for Network-based Systems

Hardware, software, and network technologies for distributed computing system design and applications.

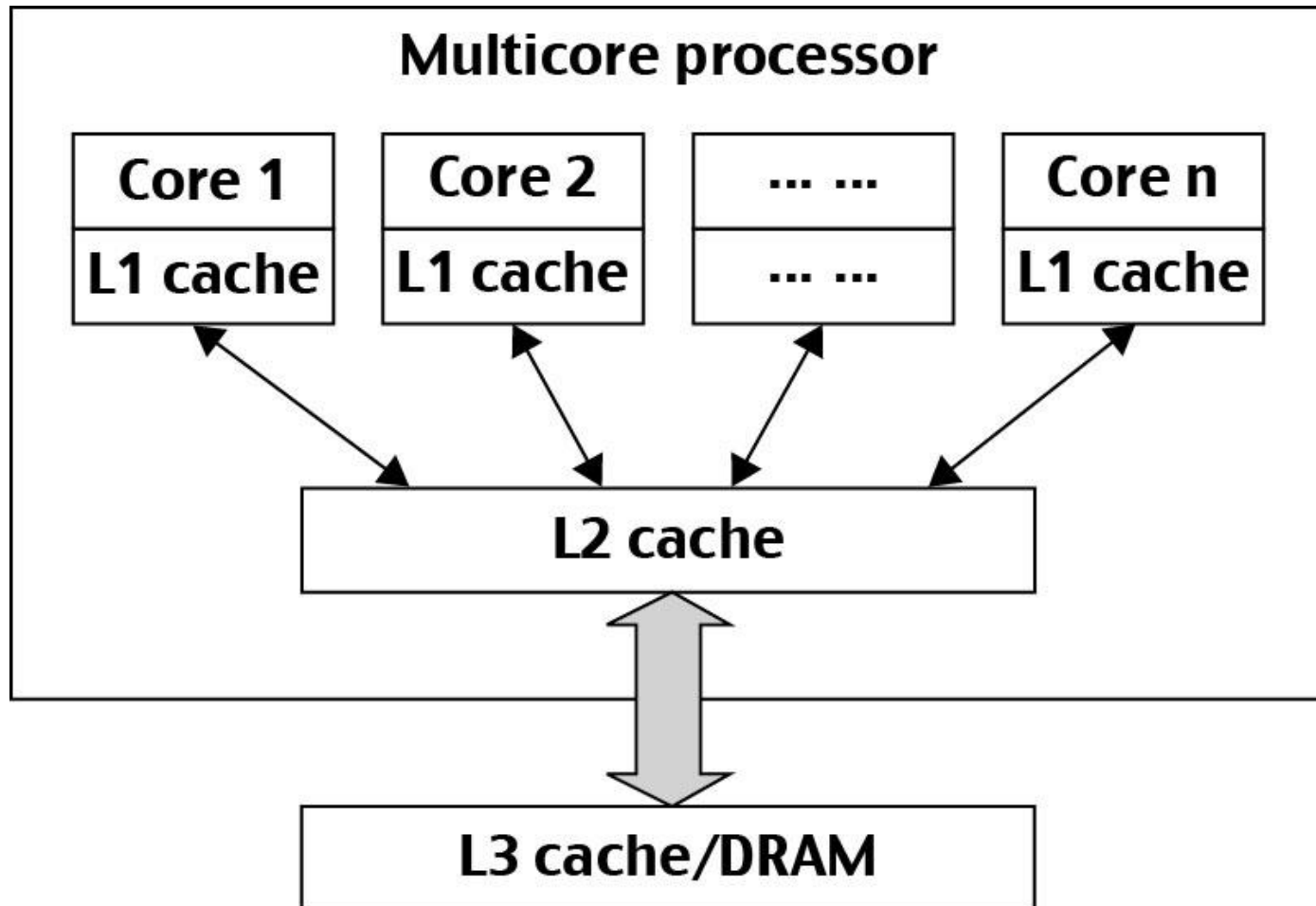
Approaches to building distributed operating systems for handling massive parallelism in a distributed environment.

Multicore CPUs and Multithreading Technologies

33 year Improvement in Processor and Network Technologies



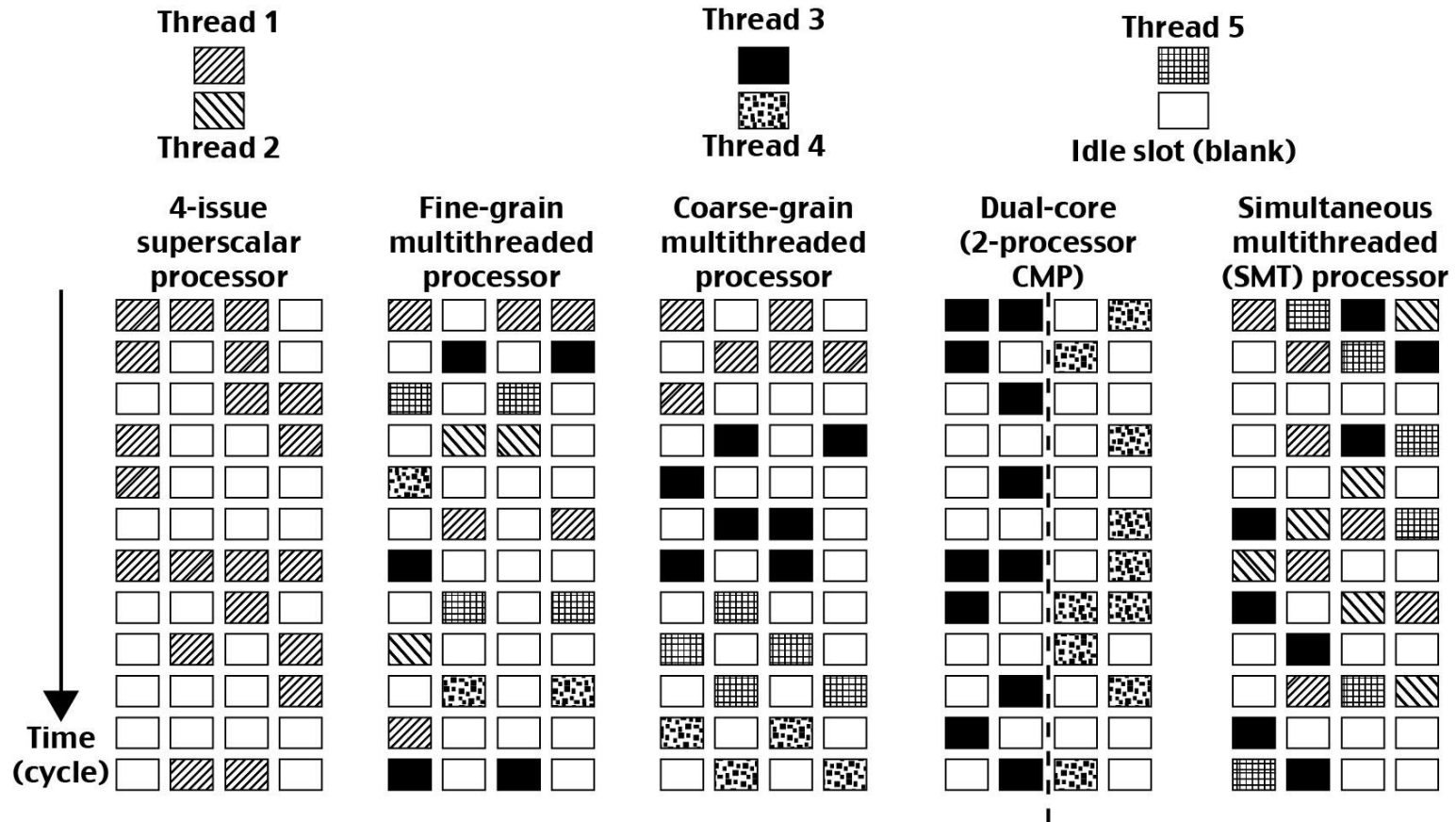
Modern Multi-core CPU Chip



Multi-threading Processors

- Four-issue superscalar (e.g. Sun Ultrasparc I)
 - Implements instruction level parallelism (ILP) within a single processor.
 - Executes more than one instruction during a clock cycle by sending multiple instructions to redundant functional units.
- Fine-grain multithreaded processor
 - Switch threads after each cycle
 - Interleave instruction execution
 - If one thread stalls, others are executed
- Coarse-grain multithreaded processor
 - Executes a single thread until it reaches certain situations
- Simultaneous multithread processor (SMT)
 - Instructions from more than one thread can execute in any given pipeline stage at a time.

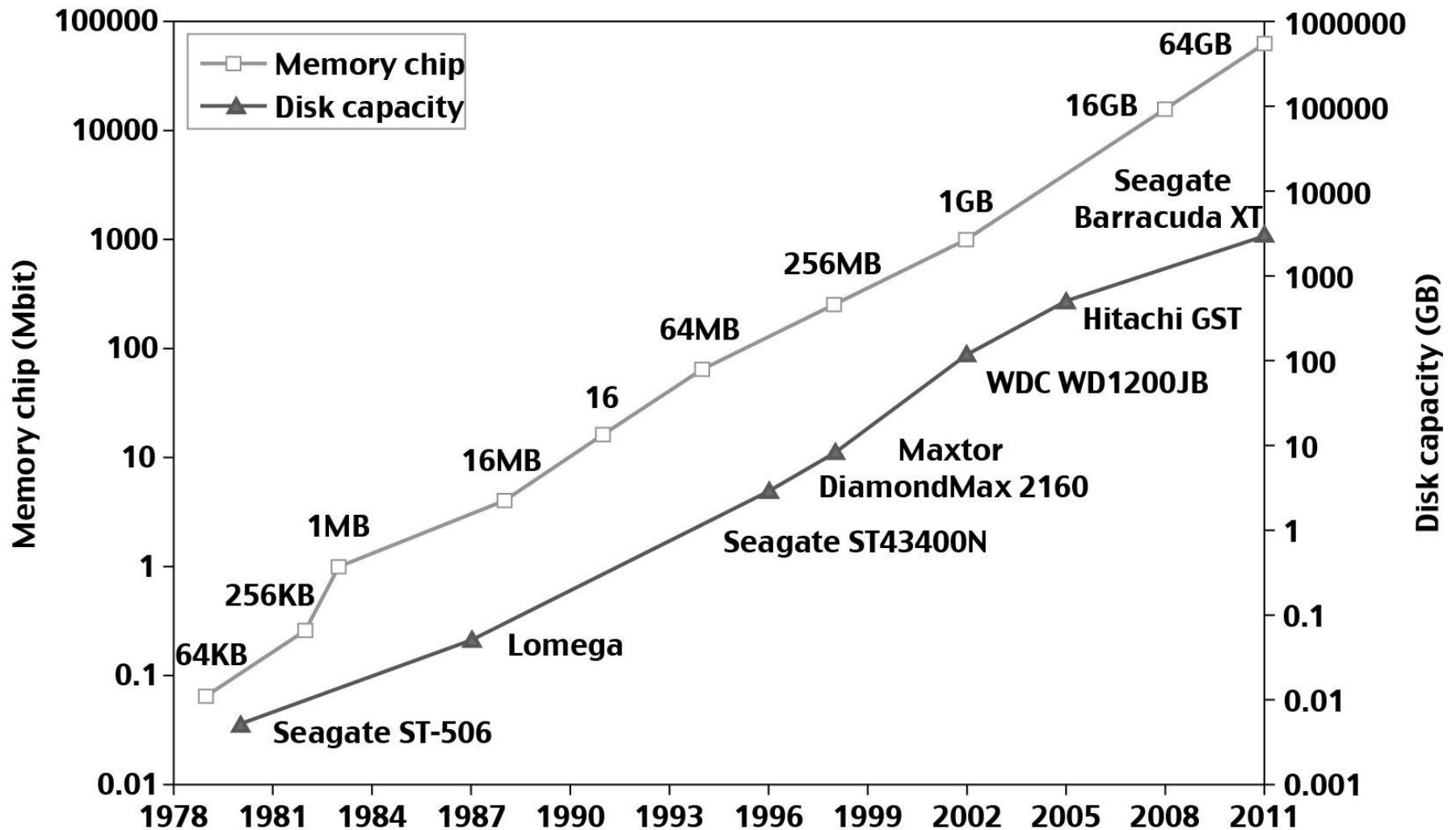
5 Micro-architectures of CPUs



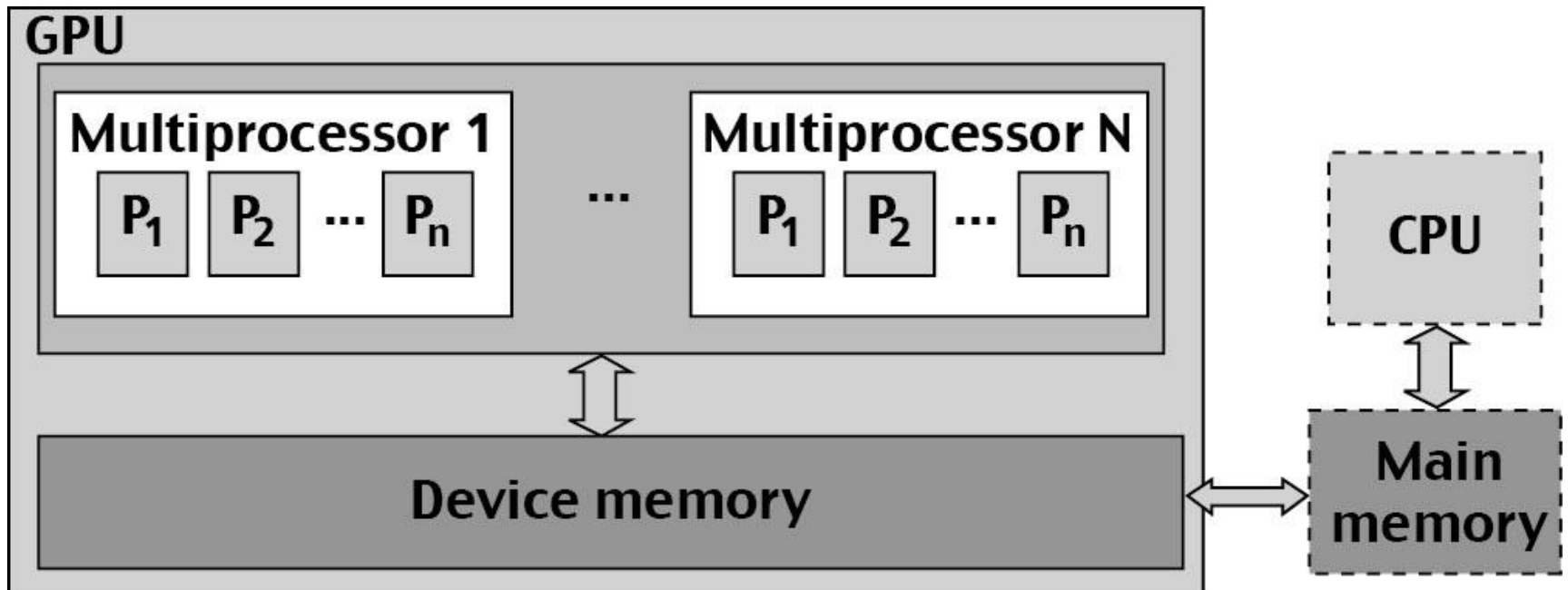
Each row represents the issue slots for a single execution cycle:

- A filled box indicates that the processor found an instruction to execute in that issue slot on that cycle;
- An empty box denotes an unused slot.

33 year Improvement in Memory and Disk Technologies

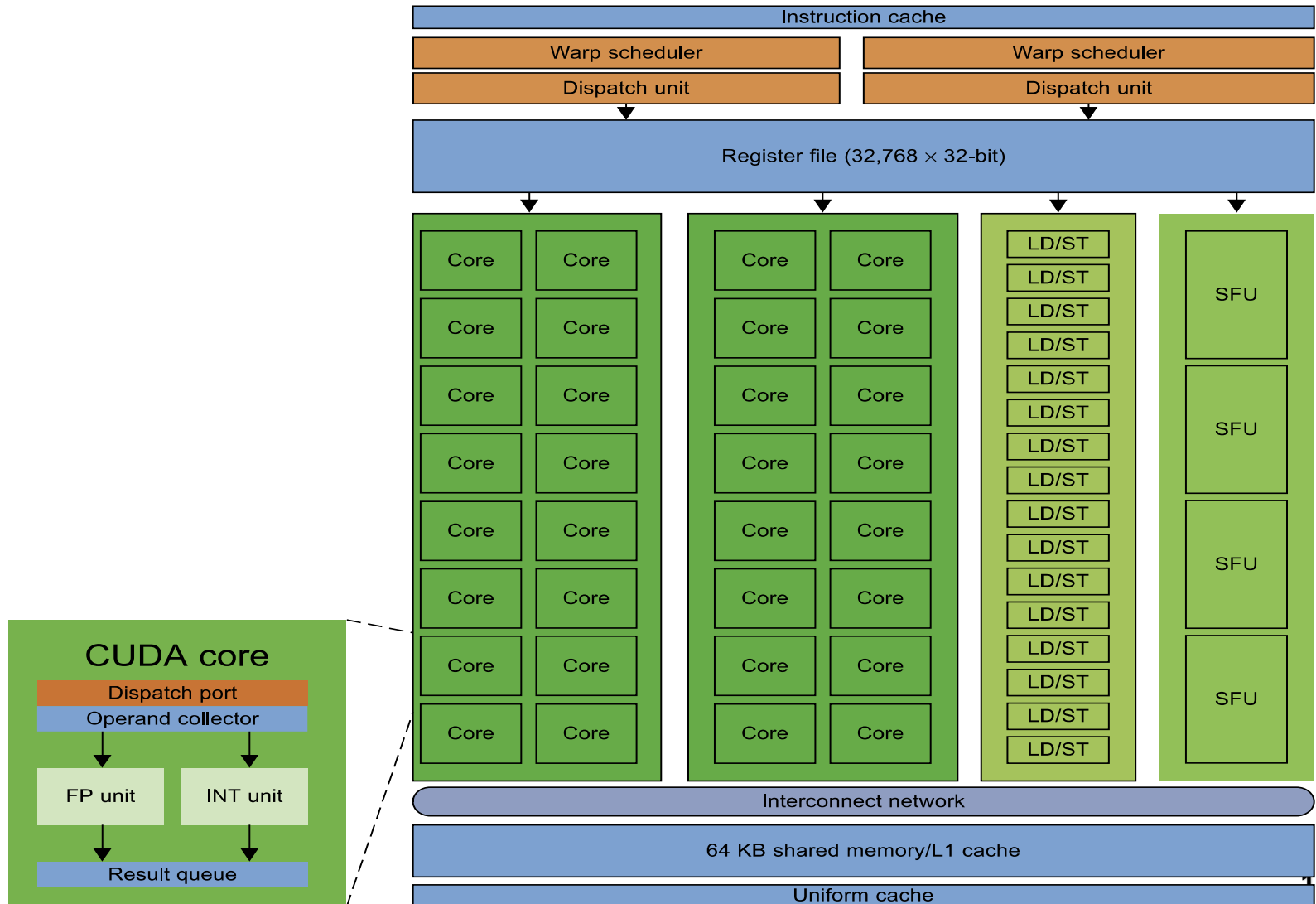


Architecture of A Many-Core Multiprocessor GPU interacting with a CPU Processor



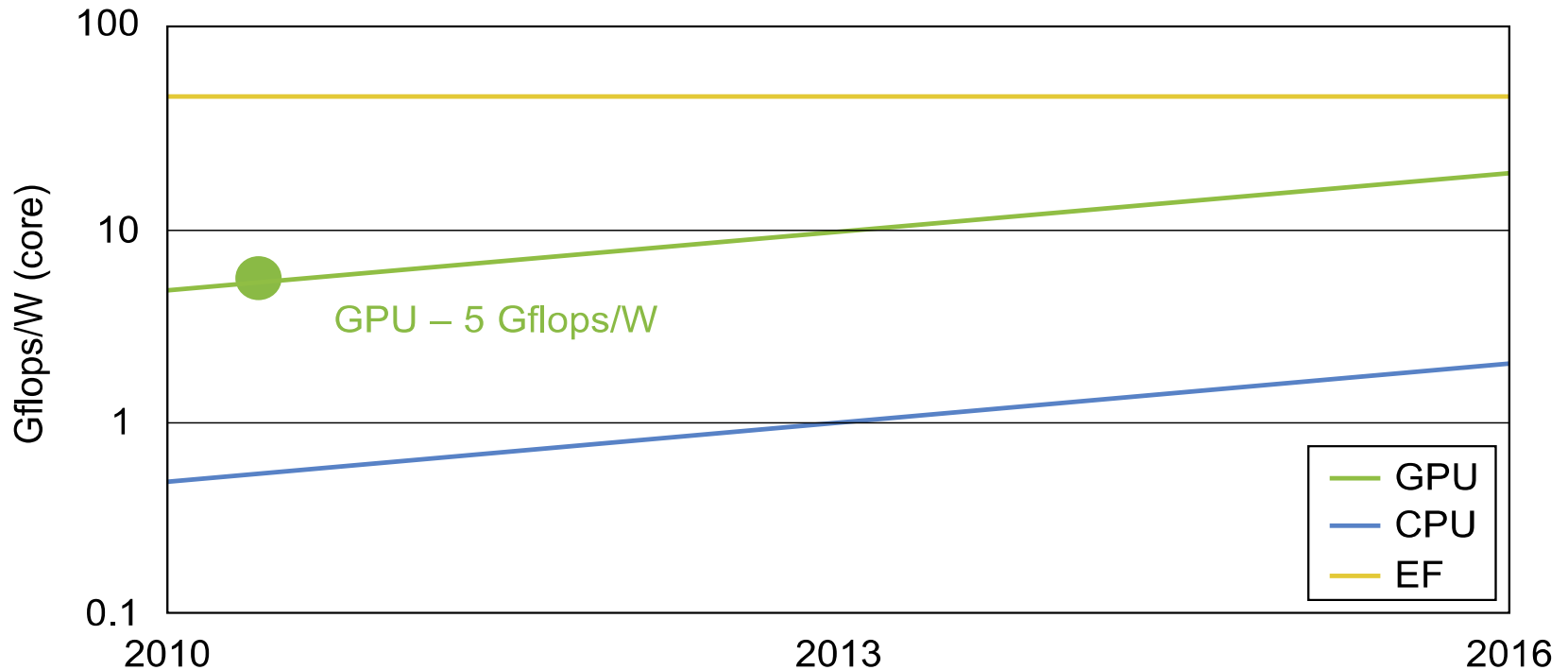
NVIDIA Fermi GPU

Built with 16 streaming multiprocessors (SMs) of 32 CUDA cores each; only one SM is shown.



GPU Performance (Power Efficiency)

The GPU performance (middle line, measured 5 Gflops/W/core), compared with the lower CPU performance (lower line measured 0.8 Gflops/W/core) and the estimated 60 Gflops/W/core performance for the Exascale (EF in upper curve) in the future.

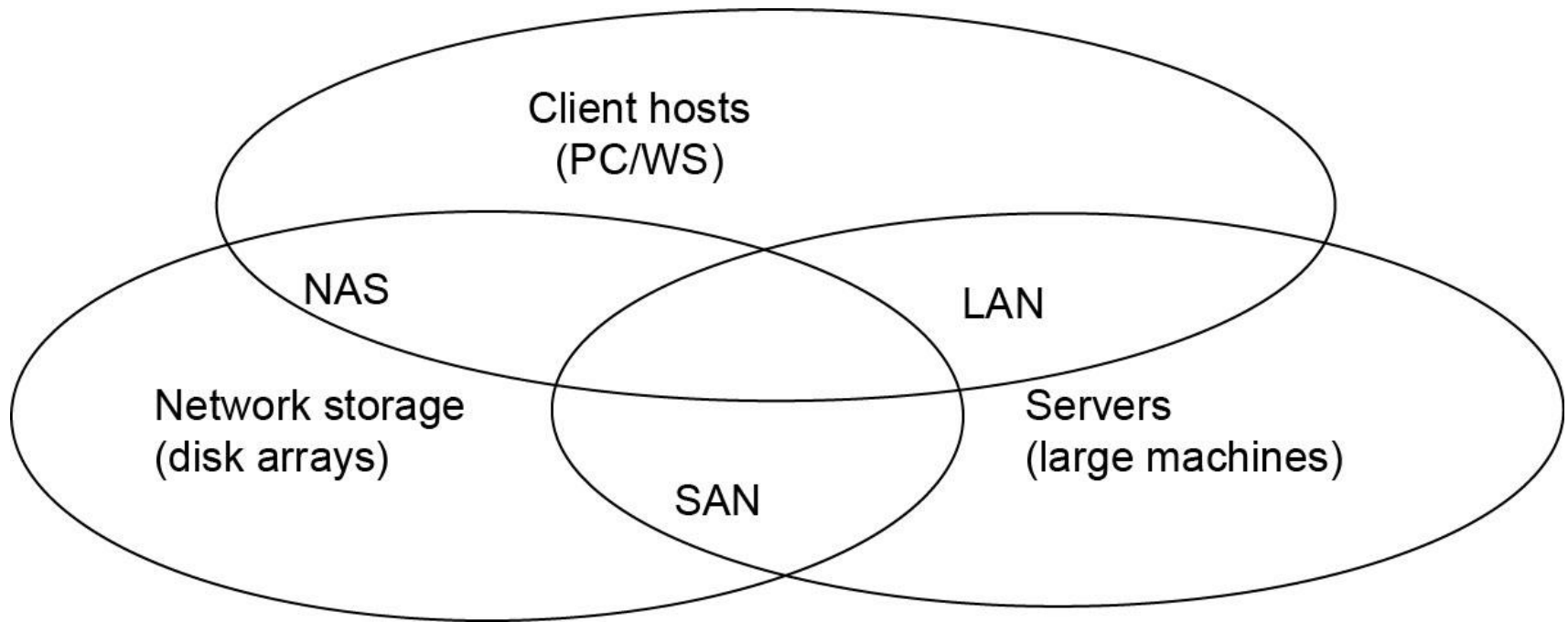


Bottom – CPU - 0.8 Gflops/W/Core

Middle – GPU - 5 Gflops/W/Core

Top - EF – Exascale computing (10^{18} Flops)

Interconnection Networks



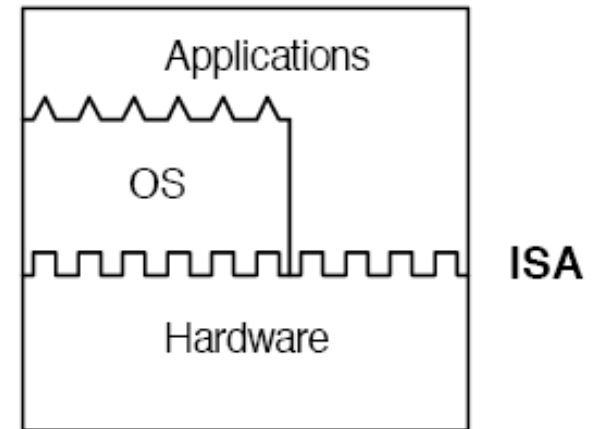
- SAN (storage area network) - connects servers with disk arrays
- LAN (local area network) – connects clients, hosts, and servers
- NAS (network attached storage) – connects clients with large storage systems

Virtual Machines and Virtualization Middleware

- Eliminate real machine constraint
 - Increases portability and flexibility
- Virtual machine adds software to a physical machine to give it the appearance of a different platform or multiple platforms.
- Benefits
 - Cross platform compatibility
 - Increase Security
 - Enhance Performance
 - Simplify software migration

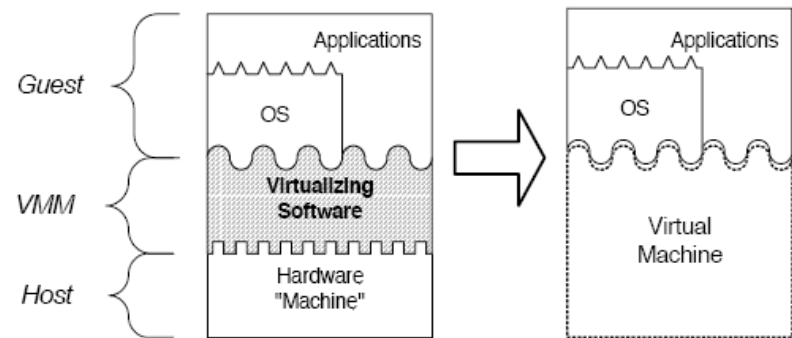
Initial Hardware Model

- All applications access hardware resources (i.e. memory, i/o) through system calls to operating system (privileged instructions)
- Advantages
 - Design is decoupled (i.e. OS people can develop OS separate of Hardware people developing hardware)
 - Hardware and software can be upgraded without notifying the Application programs
- Disadvantage
 - Application compiled on one ISA will not run on another ISA..
 - Applications compiled for Mac use different operating system calls then application designed for windows.
 - ISA's must support old software
 - Can often be inhibiting in terms of performance
 - Since software is developed separately from hardware... Software is not necessarily optimized for hardware.



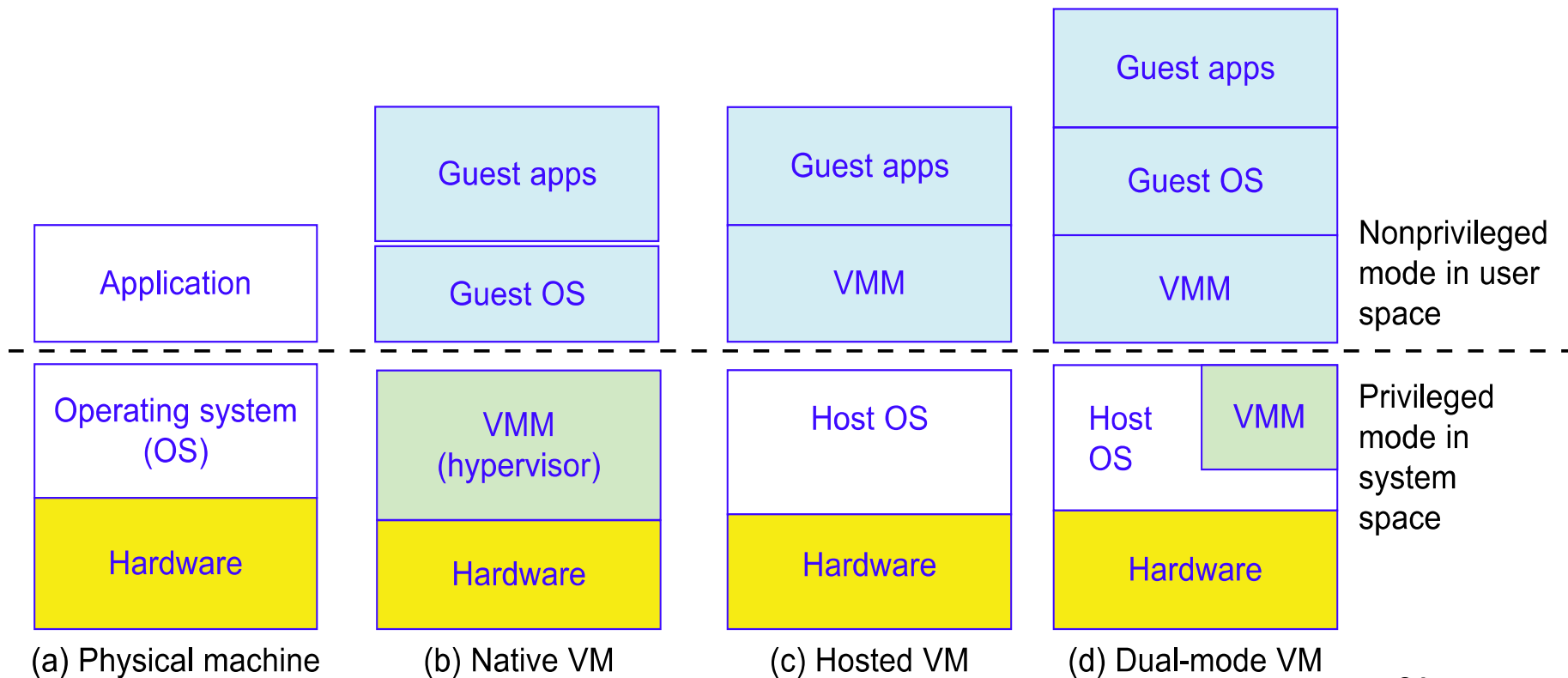
Virtual Machine Basics

- Virtual software placed between underlying machine and conventional software
 - Conventional software sees different ISA from the one supported by the hardware
- Virtualization process involves:
 - Mapping of virtual resources (registers and memory) to real hardware resources
 - Using real machine instructions to carry out the actions specified by the virtual machine instructions



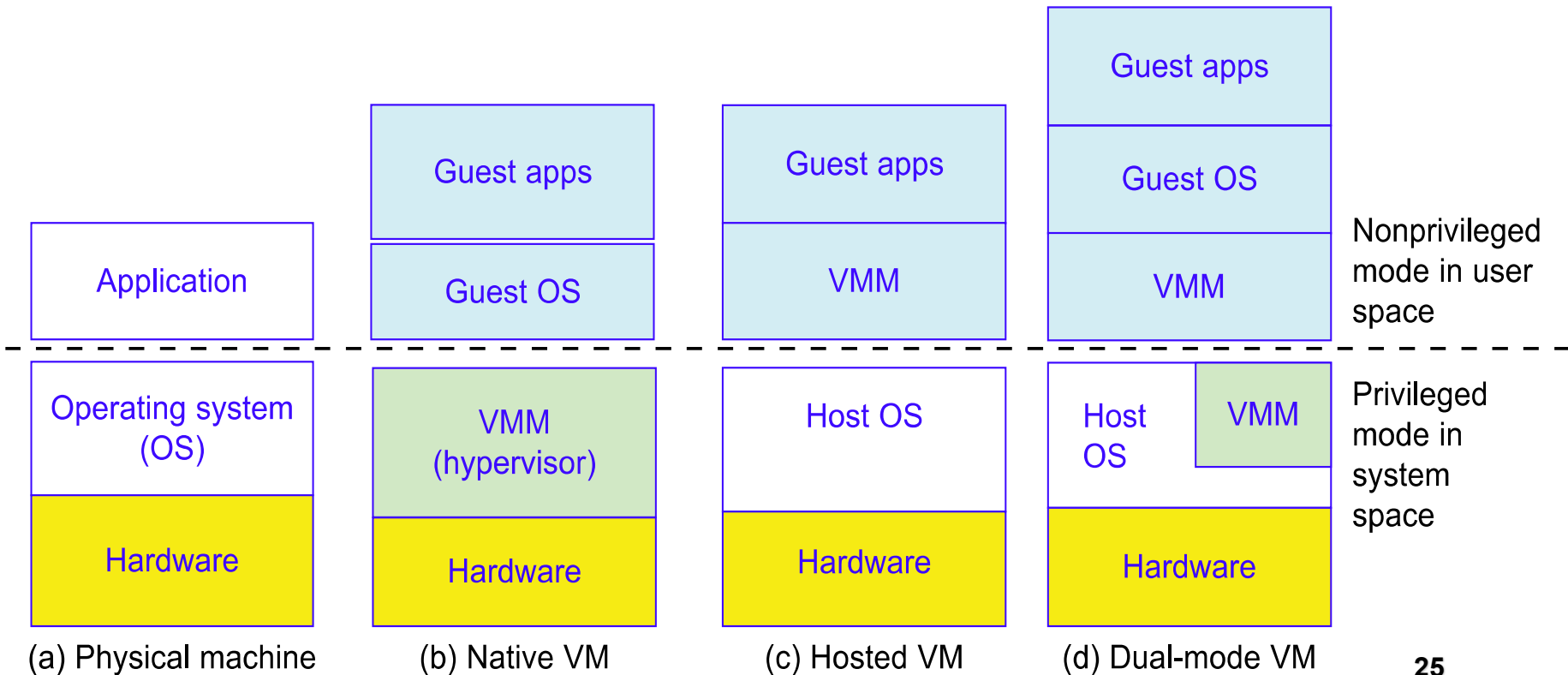
Three VM Architectures

- Part (a) the host machine is equipped with physical hardware such as an x-86 architecture desktop running its installed Windows OS.
- Part (b) a native VM installed with the use of a VMM called a hypervisor (bare-metal VM) in privileged mode.
 - The guest OS could be a Linux system and the hypervisor is the XEN system



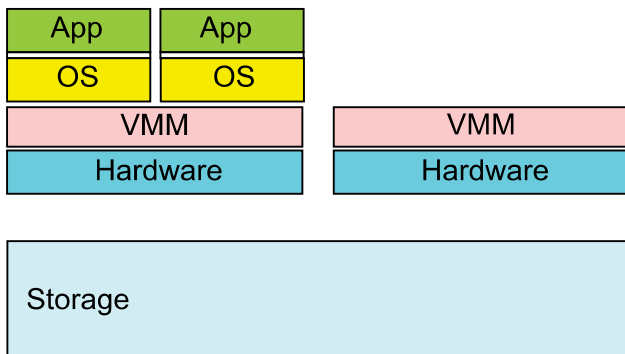
Three VM Architectures

- Part (a) host VM where the VMM runs in nonprivileged mode. The host OS need not be modified.
- Part (d) VM dual mode, part of the VMM runs at the user level and another part runs at the supervisor level.
 - The host OS may have to be modified to some extent. Multiple VMs can be ported to a given hardware system to support the virtualization process.

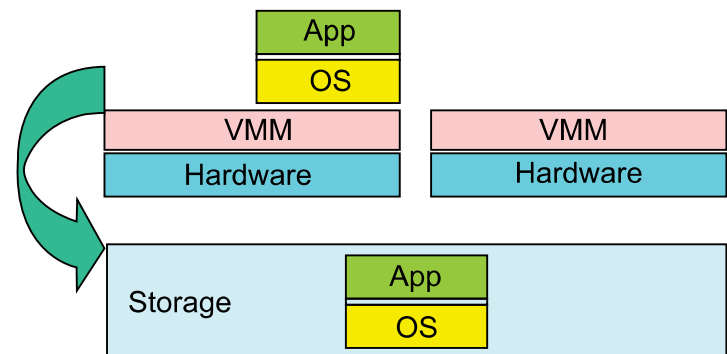


Low-level VMM operations

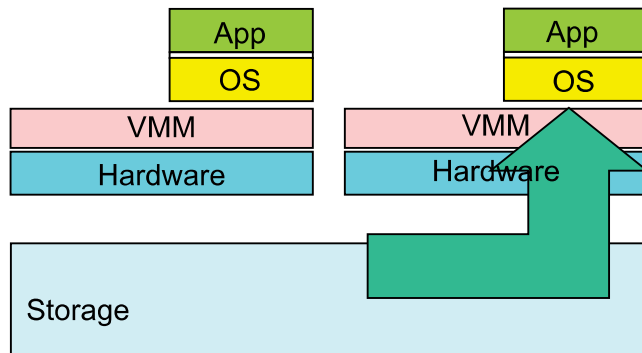
- First, the VMs can be multiplexed between hardware machines.
- Second, a VM can be suspended and stored in stable storage.
- Third, a suspended VM can be resumed or provisioned to a new hardware platform.
- Finally, a VM can be migrated from one hardware platform to another.



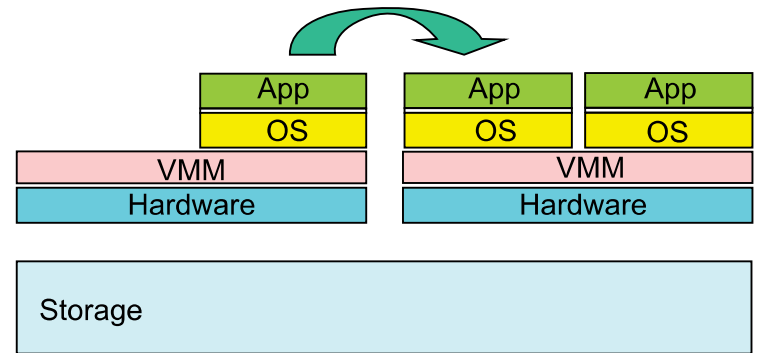
(a) Multiplexing



(b) Suspension (storage)



(c) Provision (resume)



(d) Live migration

Virtual Infrastructures

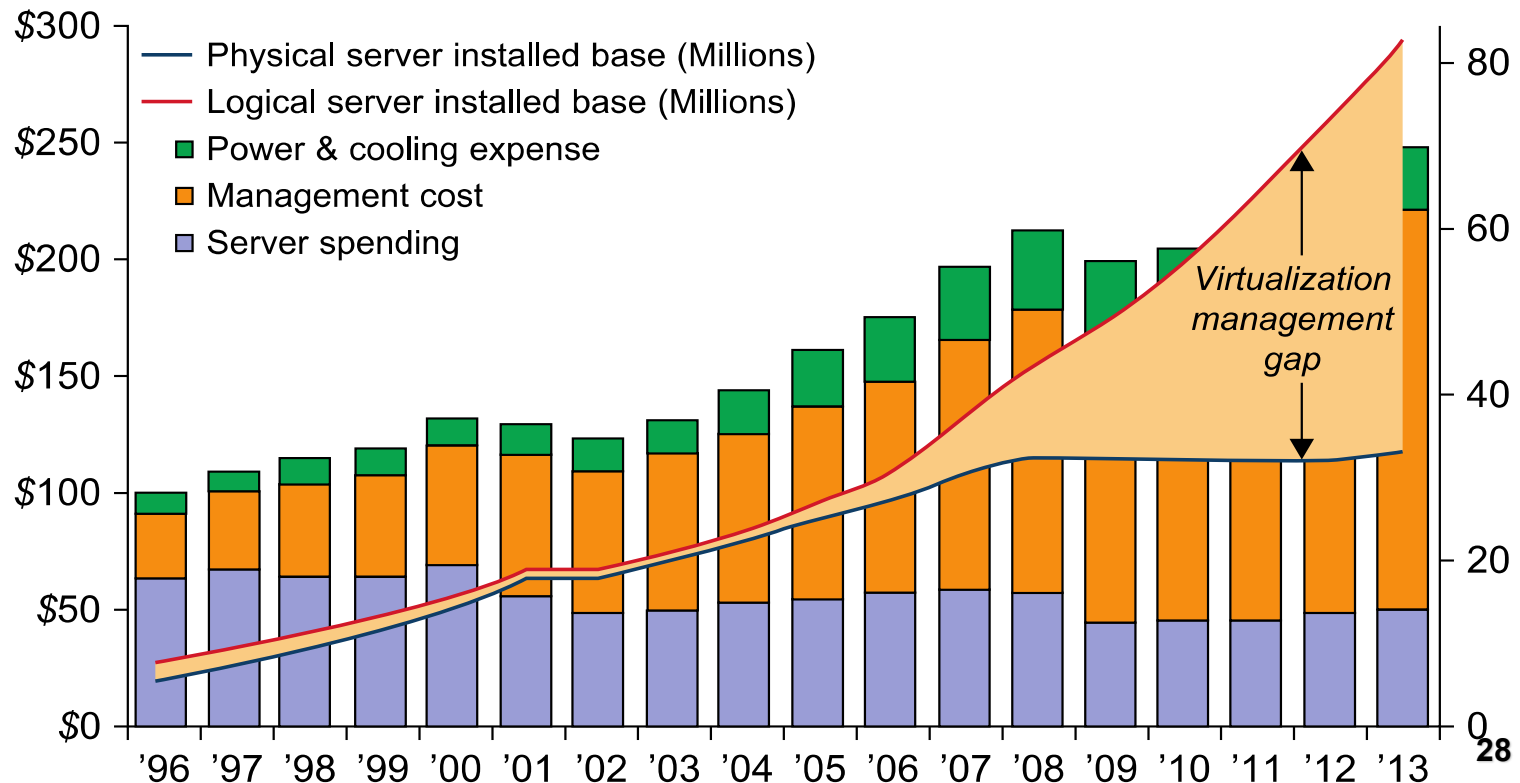
- Virtual infrastructure is what connects resources to distributed applications.
- It is a dynamic mapping of system resources to specific applications.
- The result is decreased costs and increased efficiency and responsiveness.
- Virtualization for **server consolidation** and **containment** is a good example of this.

Datacenter Virtualization and Server Cost Distribution

- Data center design emphasizes the performance/price ratio over speed performance alone.
 - storage and energy efficiency are more important than sheer speed performance.
- Figure shows server growth and cost breakdown of data centers over 15 years.
 - Worldwide, about 43 million servers are in use as of 2010.
 - The cost of utilities exceeds the cost of hardware after three years.

Customer spending (\$B)

Millions installed servers

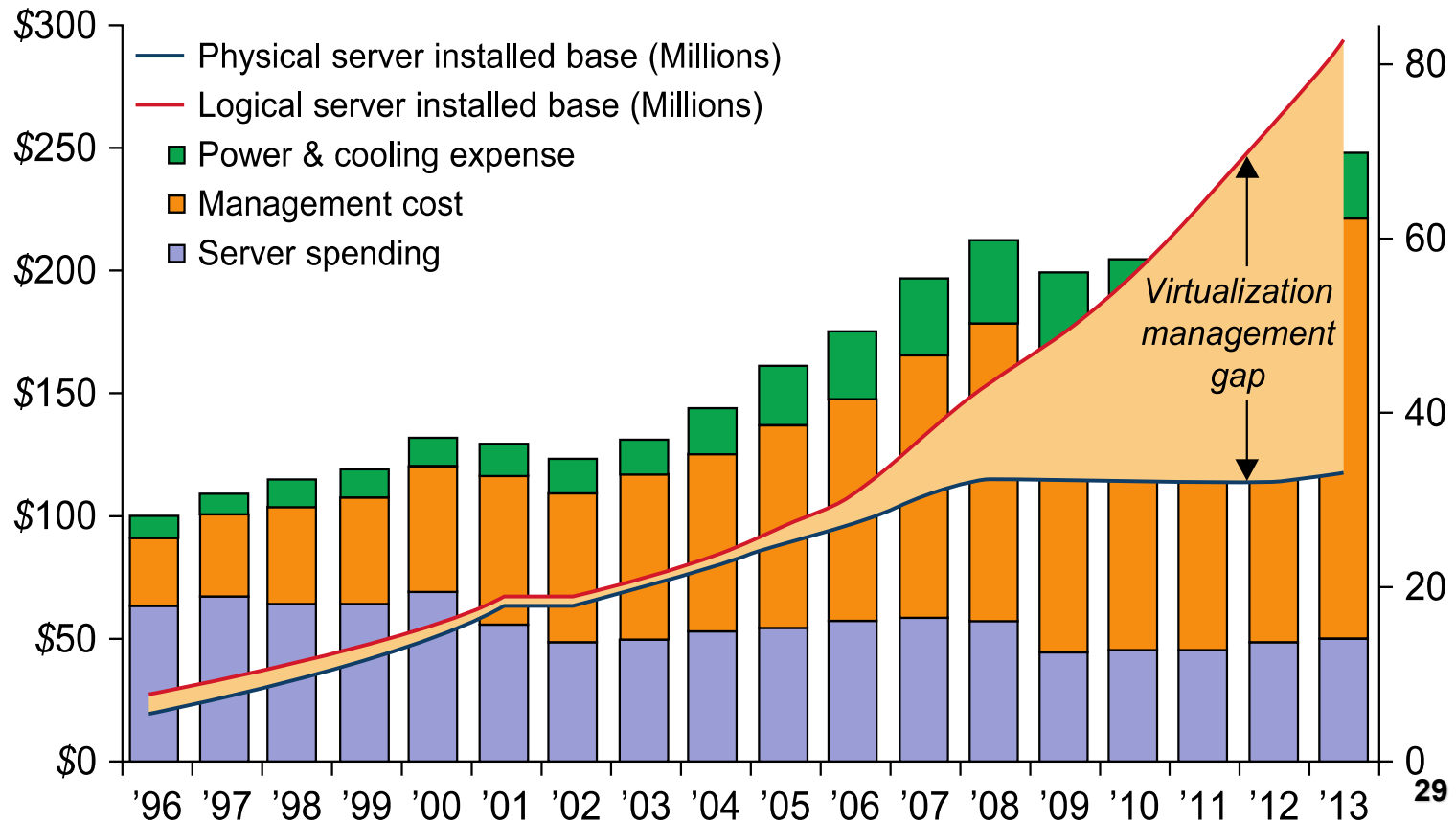


Datacenter Virtualization and Server Cost Distribution

- 40 percent of data center costs in total goes toward purchasing IT equipment and power and cooling expenses.
- 60 percent of the cost to run a data center is allocated to management and maintenance.

Customer spending (\$B)

Millions installed servers



System Models for Distributed and Cloud Computing

- Distributed and cloud computing systems are built over a large number of autonomous computer nodes.
- These nodes are interconnected by SANs, LANs, or WANs in a hierarchical manner.
- A WAN can connect many local clusters to form a very large cluster of clusters.
- A massive system with millions of computers connected to edge networks can be built.
- Massive systems are classified into four groups:
 - clusters,
 - P2P networks,
 - computing grids, and
 - Internet clouds over huge data centers.

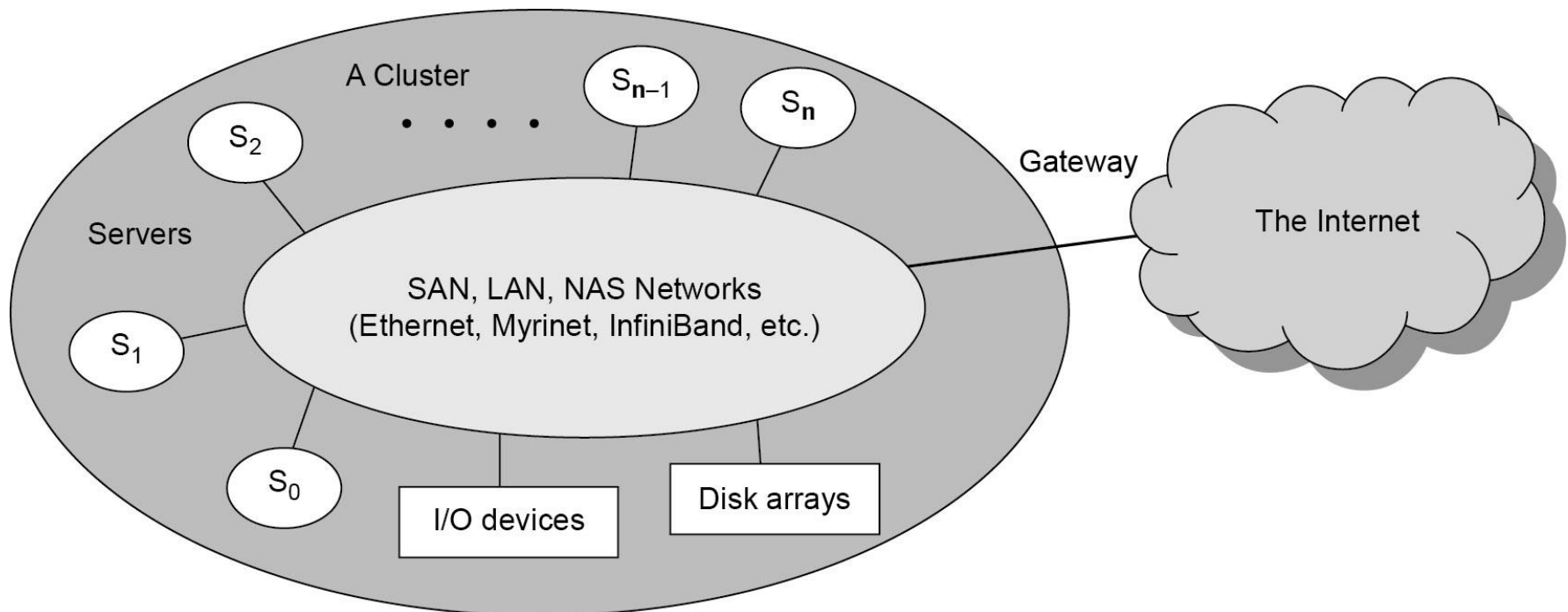
System Models for Distributed and Cloud Computing

Table 1.2 Classification of Distributed Parallel Computing Systems

Functionality, Applications	Multicomputer Clusters [27, 33]	Peer-to-Peer Networks [40]	Data/Computational Grids [6, 42]	Cloud Platforms [1, 9, 12, 17, 29]
Architecture, Network Connectivity and Size	Network of compute nodes interconnected by SAN, LAN, or WAN, hierarchically	Flexible network of client machines logically connected by an overlay network	Heterogeneous clusters interconnected by high-speed network links over selected resource sites.	Virtualized cluster of servers over datacenters via service-level agreement
Control and Resources Management	Homogeneous nodes with distributed control, running Unix or Linux	Autonomous client nodes, free in and out, with distributed self-organization	Centralized control, server oriented with authenticated security, and static resources	Dynamic resource provisioning of servers, storage, and networks over massive datasets
Applications and network-centric services	High-performance computing, search engines, and web services, etc.	Most appealing to business file sharing, content delivery, and social networking	Distributed super-computing, global problem solving, and datacenter services	Upgraded web search, utility computing, and outsourced computing services
Representative Operational Systems	Google search engine, SunBlade, IBM Road Runner, Cray XT4, etc.	Gnutella, eMule, BitTorrent, Napster, KaZaA, Skype, JXTA, and .NET	TeraGrid, GriPhyN, UK EGEE, D-Grid, ChinaGrid, etc.	Google App Engine, IBM Bluecloud, Amazon Web Service(AWS), and Microsoft Azure,

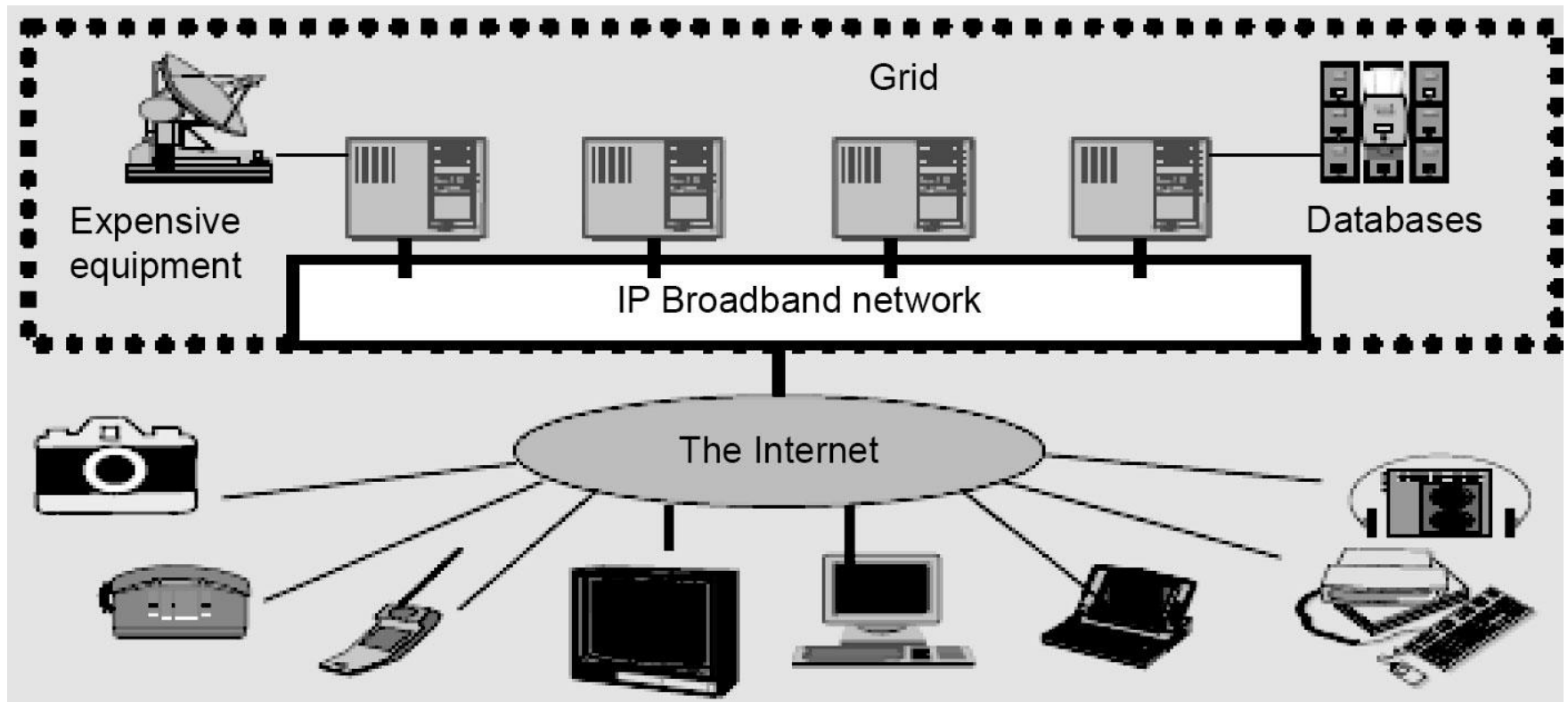
A Typical Cluster Architecture

A cluster of servers interconnected by a high-bandwidth SAN or LAN with shared I/O devices and disk arrays; the cluster acts as a single computer attached to the Internet.



Computational or Data Grid

Computational grid or data grid providing computing utility, data, and information services through resource sharing and cooperation among participating organizations.



A Typical Computational Grid

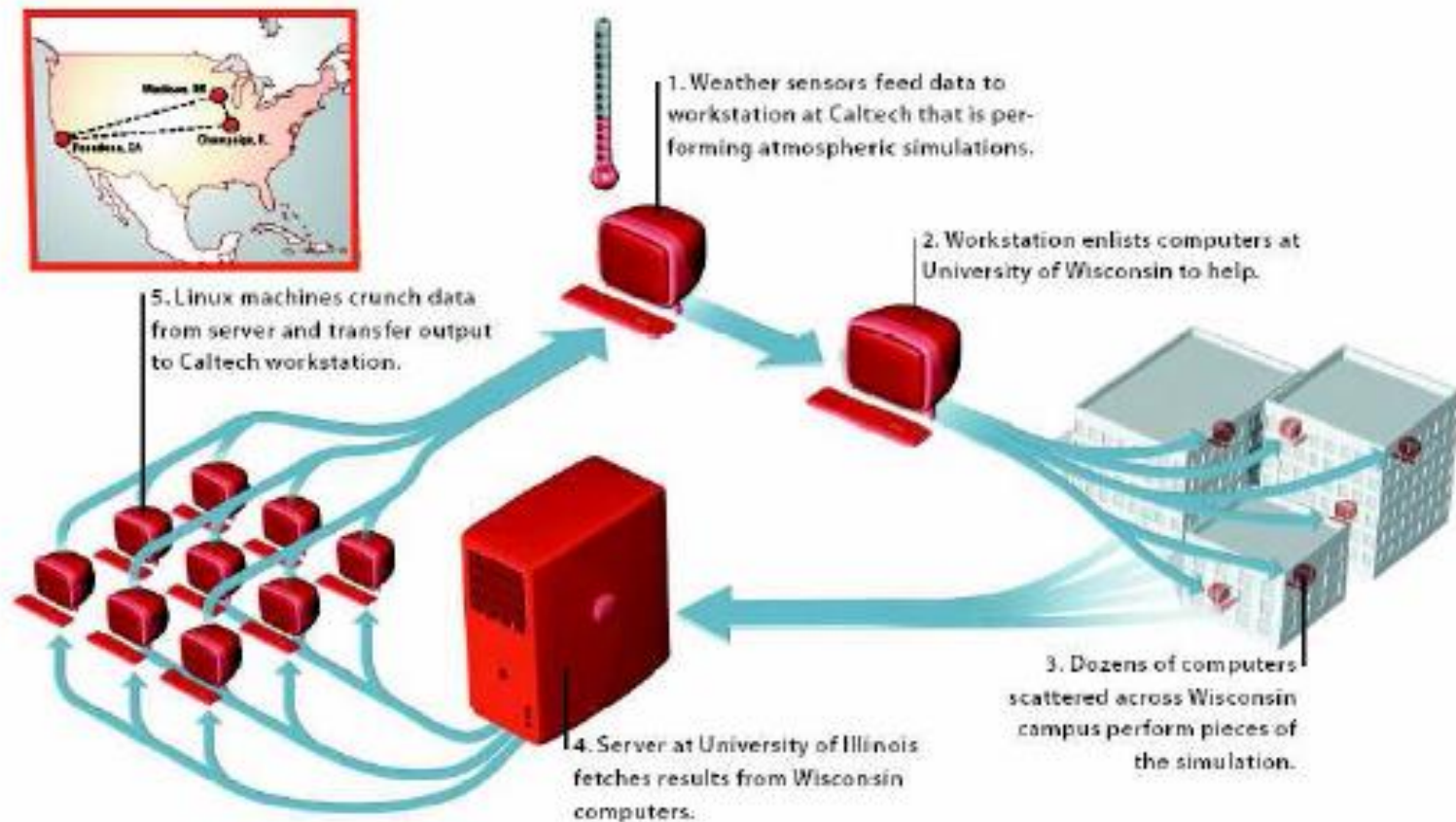
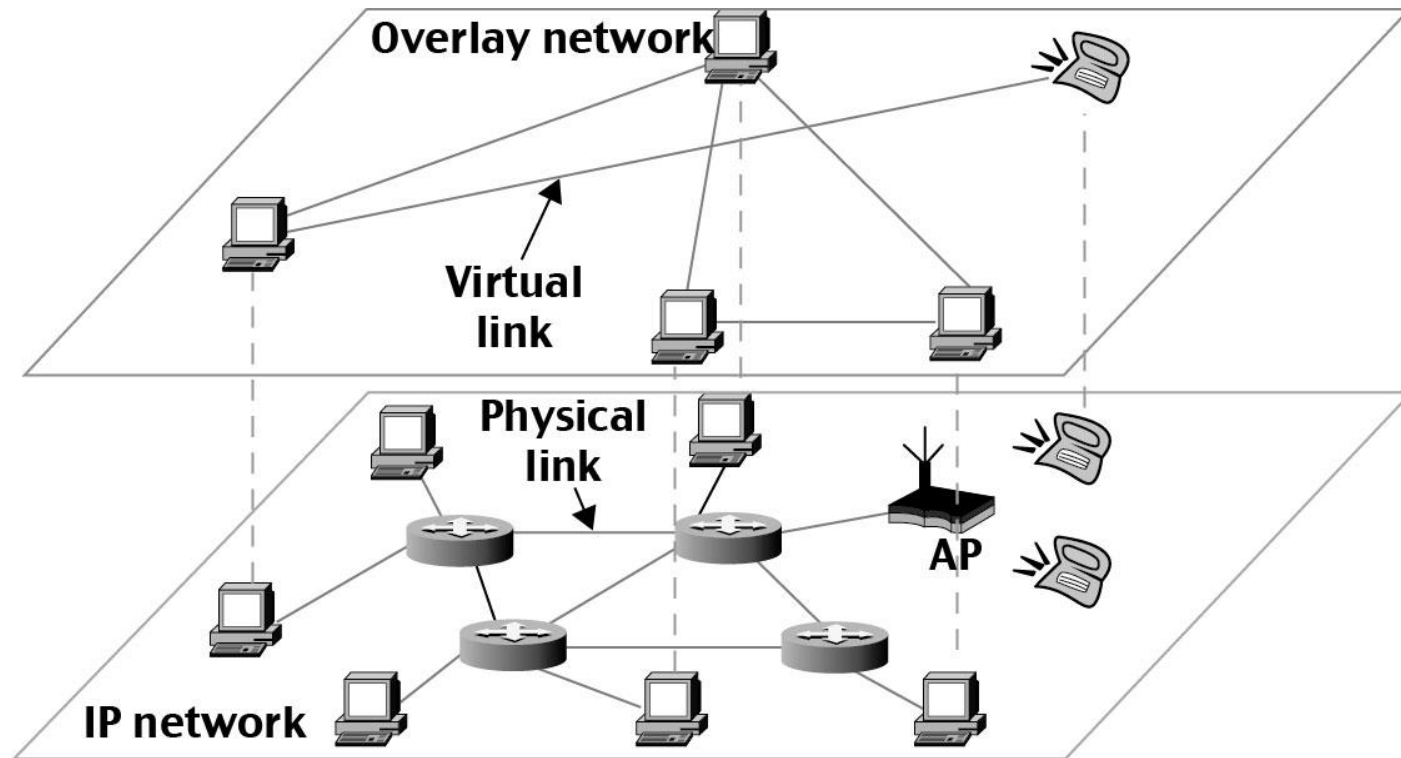


Figure 1.17 An example computational Grid built over specialized computers at three resource sites at Wisconsin, Caltech, and Illinois. (Courtesy of Michel Waldrop, "Grid Computing", IEEE Computer Magazine, 2000. [42])

Peer-to-Peer (P2P) Network

- A distributed system architecture
- Each computer in the network can act as a client or server for other network computers.
- No centralized control
- Typically many nodes, but unreliable and heterogeneous
- Nodes are symmetric in function
- Take advantage of distributed, shared resources (bandwidth, CPU, storage) on peer-nodes
- Fault-tolerant, self-organizing
- Operate in dynamic environment, frequent join and leave is the norm

Peer-to-Peer (P2P) Network



- computer network built on top of another network.

- Nodes in the overlay can be thought of as being connected by virtual or logical links, each of which corresponds to a path, perhaps through many physical links, in the underlying network.
- For example, distributed systems such as cloud computing, peer-to-peer networks, and client-server applications are overlay networks because their nodes run on top of the Internet.

Peer-to-Peer (P2P) Network

Table 1.5 Major Categories of P2P Network Families [42]

System Features	Distributed File Sharing	Collaborative Platform	Distributed P2P Computing	P2P Platform
Attractive Applications	Content distribution of MP3 music, video, open software, etc.	Instant messaging, collaborative design and gaming	Scientific exploration and social networking	Open networks for public resources
Operational Problems	Loose security and serious online copyright violations	Lack of trust, disturbed by spam, privacy, and peer collusion	Security holes, selfish partners, and peer collusion	Lack of standards or protection protocols
Example Systems	Gnutella, Napster, eMule, BitTorrent, Aimster, KaZaA, etc.	ICQ, AIM, Groove, Magi, Multiplayer Games, Skype, etc.	SETI@home, Geonome@home, etc.	JXTA, .NET, FightingAid@home, etc.

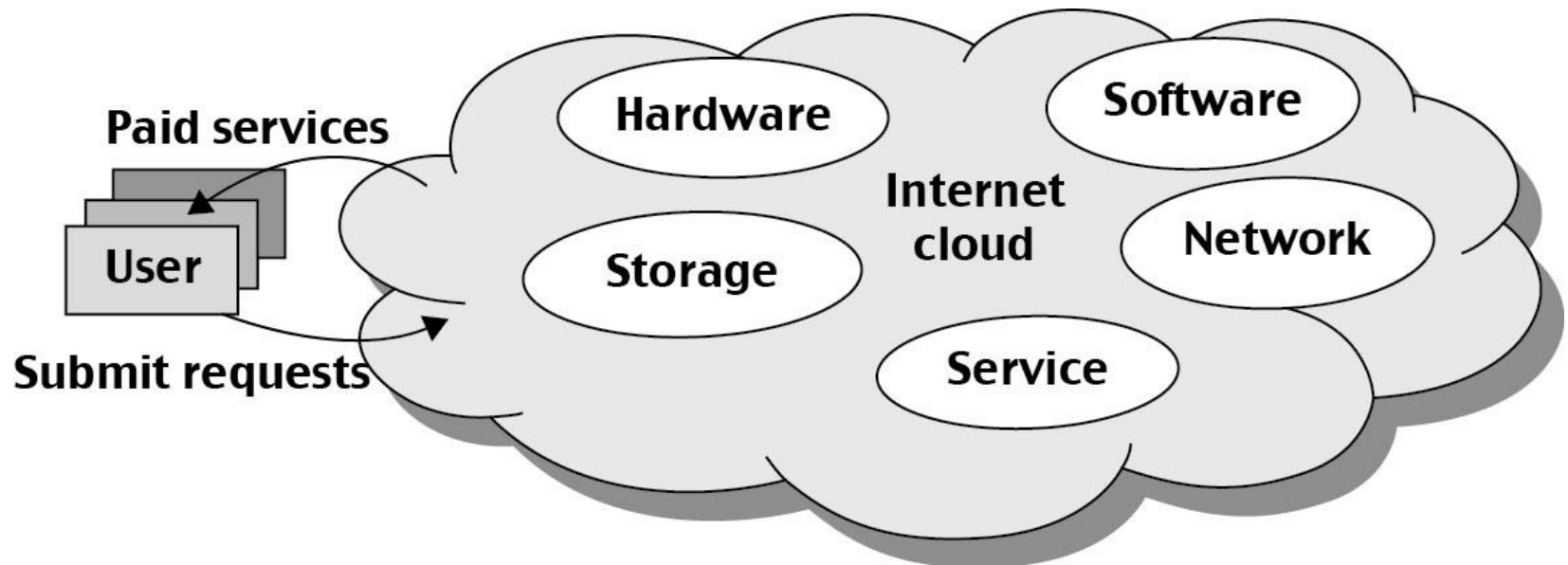
Cloud Computing over the Internet

- Historical roots in today's Internet apps
 - Search, email, social networks
 - File storage (Live Mesh, Mobile Me, Flickr, ...)
- A cloud infrastructure provides a framework to manage scalable, reliable, on-demand access to applications
- A cloud is the “invisible” backend to many of our mobile applications
- A model of computation and data storage based on “pay as you go” access to “unlimited” remote data center capabilities



Basic Concept of Internet Clouds

- **Cloud computing** is the use of computing resources (hardware and software) that are delivered as a service over a network (typically the Internet).
- The name comes from the use of a cloud-shaped symbol as an abstraction for the complex infrastructure it contains in system diagrams.
- Cloud computing entrusts remote services with a user's data, software and computation.



The Next Revolution in IT Cloud Computing

• Classical Computing

- Buy & Own
 - Hardware, System Software, Applications often to meet peak needs.
- Install, Configure, Test, Verify, Evaluate
- Manage
- ..
- Finally, use it
- \$\$\$\$....\$(High CapEx)

• Cloud Computing

- Subscribe
- Use



- \$ - pay for what you use, based on QoS

Cloud Service Models (1)

Infrastructure as a service (IaaS)

- Most basic cloud service model
- Cloud providers offer computers, as physical or more often as virtual machines, and other resources.
- Virtual machines are run as guests by a hypervisor, such as Xen or KVM.
- Cloud users deploy their applications by then installing operating system images on the machines as well as their application software.
- Cloud providers typically bill IaaS services on a utility computing basis, that is, cost will reflect the amount of resources allocated and consumed.
- Examples of IaaS include: Amazon CloudFormation (and underlying services such as Amazon EC2), Rackspace Cloud, Terremark, and Google Compute Engine.

Cloud Service Models (2)

Platform as a service (PaaS)

- Cloud providers deliver a computing platform typically including operating system, programming language execution environment, database, and web server.
- Application developers develop and run their software on a cloud platform without the cost and complexity of buying and managing the underlying hardware and software layers.
- Examples of PaaS include: Amazon Elastic Beanstalk, Cloud Foundry, Heroku, Force.com, EngineYard, Mendix, Google App Engine, Microsoft Azure and OrangeScape.

Cloud Service Models (3)

Software as a service (SaaS)

- Cloud providers install and operate application software in the cloud and cloud users access the software from cloud clients.
- The pricing model for SaaS applications is typically a monthly or yearly flat fee per user, so price is scalable and adjustable if users are added or removed at any point.
- Examples of SaaS include: Google Apps, innkeypos, Quickbooks Online, Limelight Video Platform, Salesforce.com, and Microsoft Office 365.

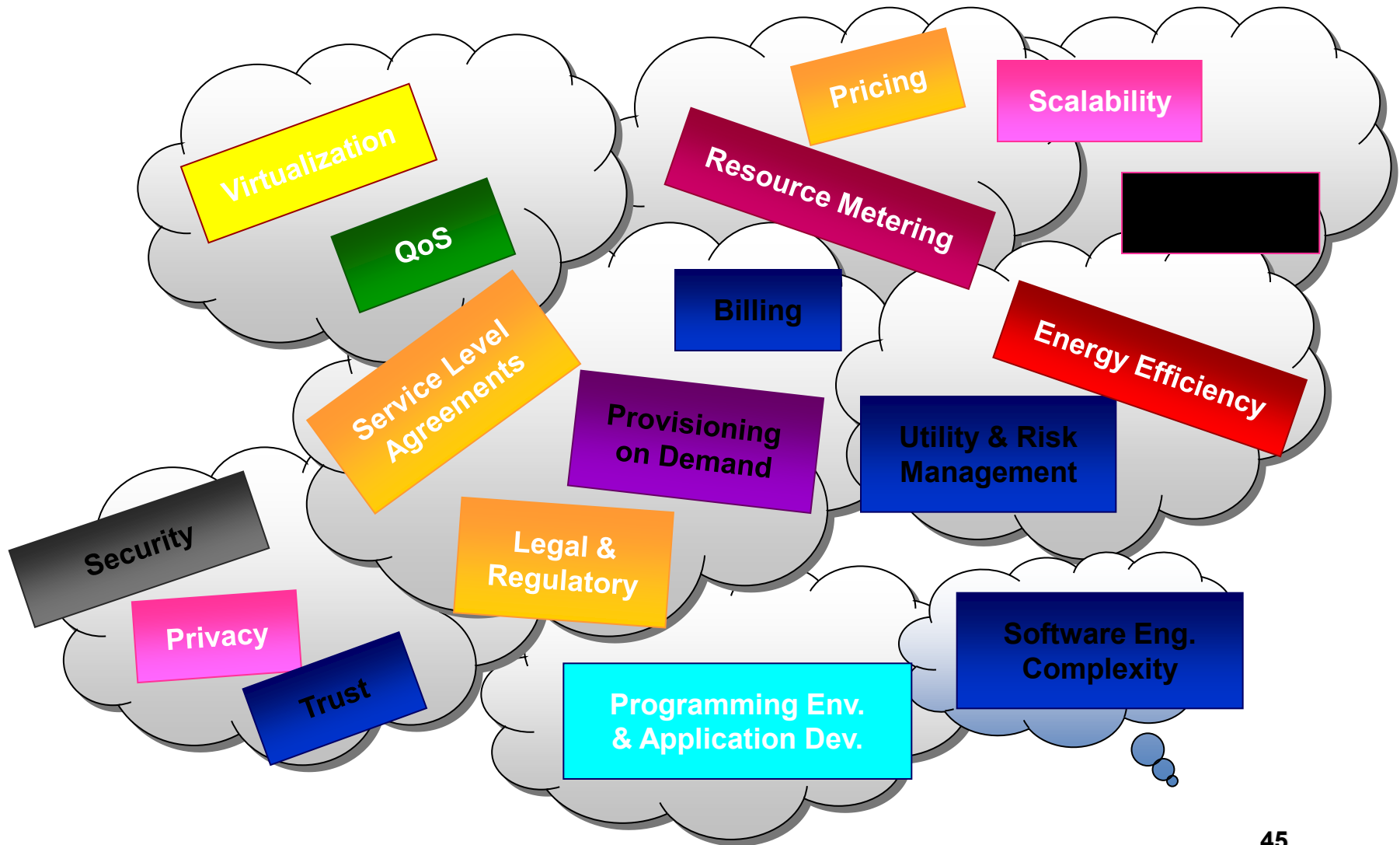
Why adapting the cloud

The following list highlights eight reasons to adapt the cloud for upgraded Internet applications and web services:

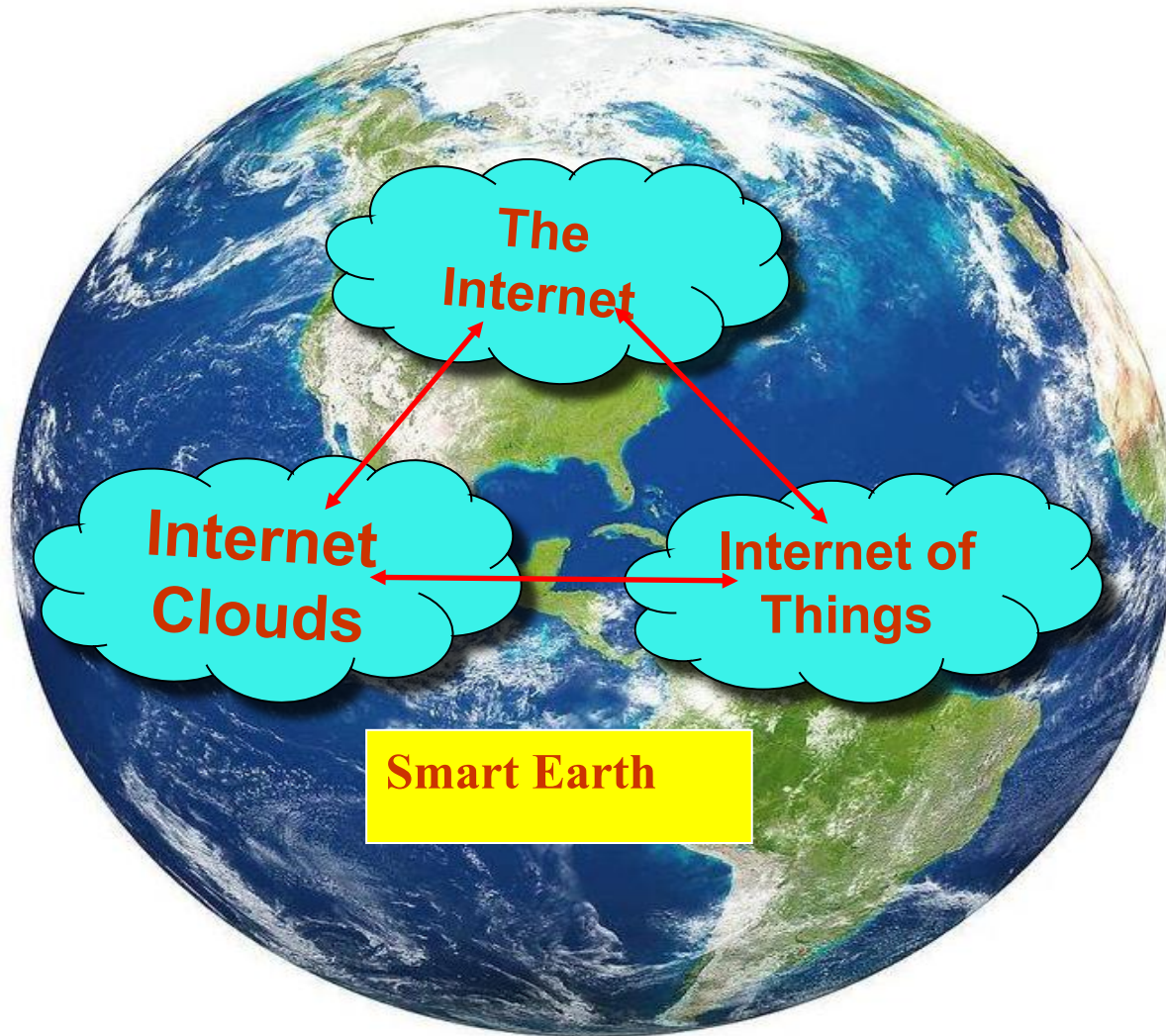
1. Desired location in areas with protected space and higher energy efficiency
2. Sharing of peak-load capacity among a large pool of users, improving overall utilization
3. Separation of infrastructure maintenance duties from domain-specific application development
4. Significant reduction in cloud computing cost, compared with traditional computing paradigms
5. Cloud computing programming and application development
6. Service and data discovery and content/service distribution
7. Privacy, security, copyright, and reliability issues
8. Service agreements, business models, and pricing policies

Cloud Computing Challenges:

Dealing with too many issues (Courtesy of R. Buyya)



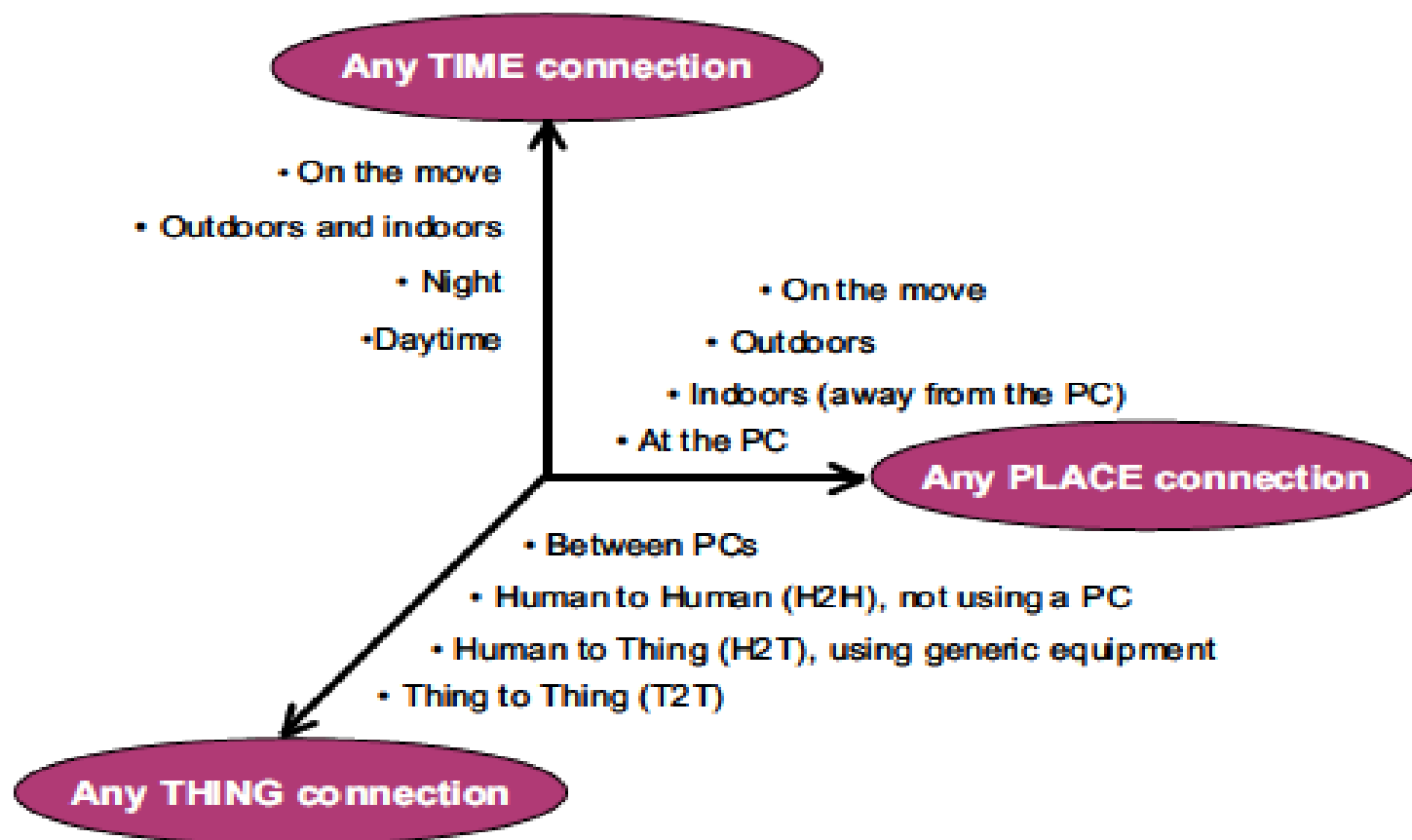
The Internet of Things (IoT)



**Smart
Earth:**

**An
IBM
Dream**

Opportunities of IoT in 3 Dimensions



Software Environments for Distributed Systems and Clouds

Popular software environments for using distributed and cloud computing systems.

- Service-oriented architecture (SOA)
- Distributed Operating Systems
- Parallel and Distributed Programming Models

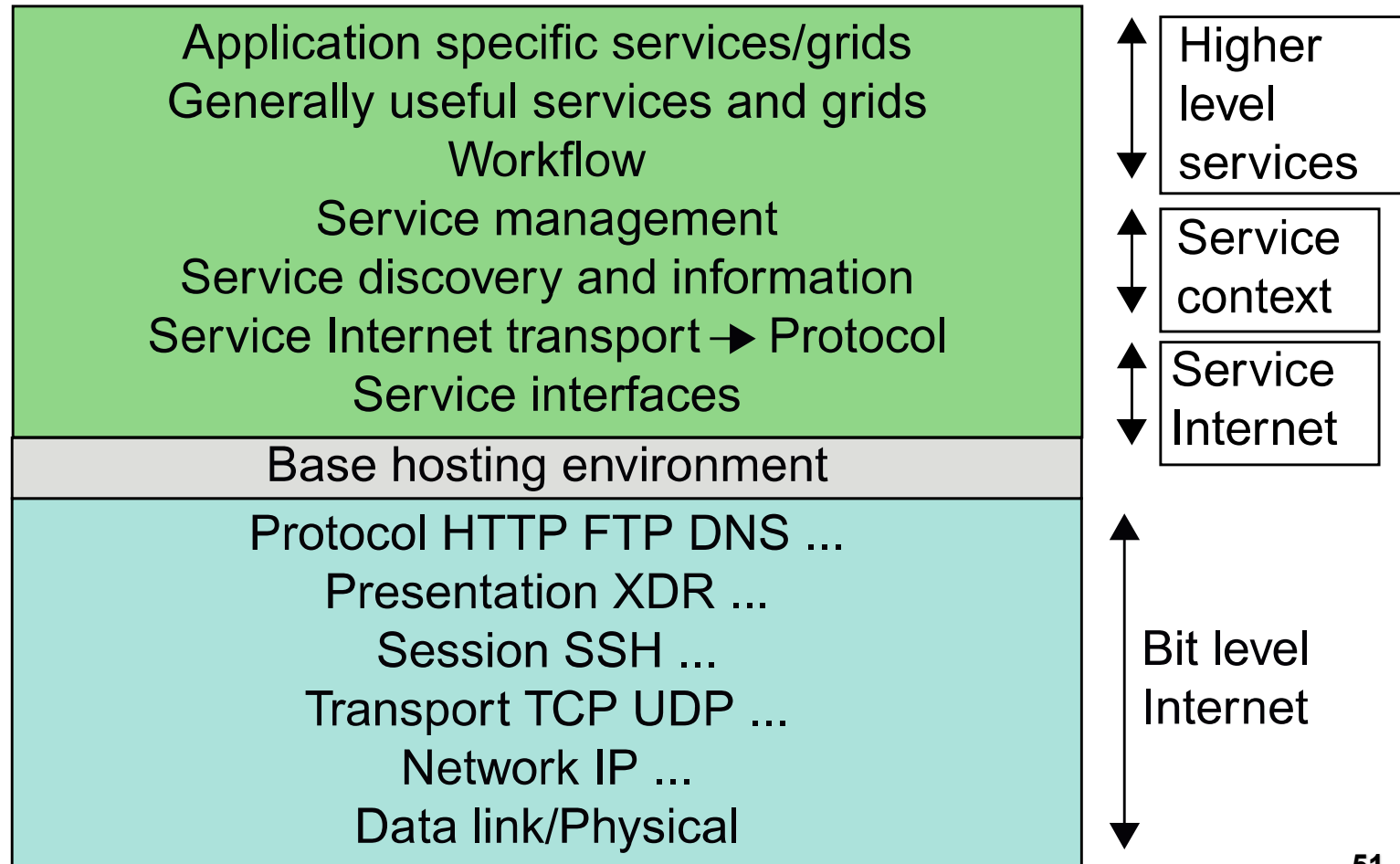
Service-oriented architecture (SOA)

- SOA is an evolution of distributed computing based on the request/reply design paradigm for synchronous and asynchronous applications.
- An application's business logic or individual functions are modularized and presented as services for consumer/client applications.
- Key to these services - their loosely coupled nature;
 - i.e., the service interface is independent of the implementation.
- Application developers or system integrators can build applications by composing one or more services without knowing the services' underlying implementations.
 - For example, a service can be implemented either in .Net or J2EE, and the application consuming the service can be on a different platform or language.

SOA key characteristics:

- SOA services have self-describing interfaces in platform-independent XML documents.
 - Web Services Description Language (WSDL) is the standard used to describe the services.
- SOA services communicate with messages formally defined via XML Schema (also called XSD).
 - Communication among consumers and providers or services typically happens in heterogeneous environments, with little or no knowledge about the provider.
 - Messages between services can be viewed as key business documents processed in an enterprise.
- SOA services are maintained in the enterprise by a registry that acts as a directory listing.
 - Applications can look up the services in the registry and invoke the service.
 - Universal Description, Definition, and Integration (UDDI) is the standard used for service registry.
- Each SOA service has a quality of service (QoS) associated with it.
 - Some of the key QoS elements are security requirements, such as authentication and authorization, reliable messaging, and policies regarding who can invoke services.

Layered Architecture for Web Services



Distributed Operating Systems

- Computers in most distributed systems are loosely coupled.
 - Thus, a distributed system inherently has multiple system images with separate OSs.
- To promote resource sharing and fast communication among node machines, it is best to have a distributed OS that manages all resources coherently and efficiently.
- Such a system is most likely to be a closed system
 - It will likely rely on message passing and RPCs for internode communications.
 - It should be pointed out that a distributed OS is crucial for upgrading the performance, efficiency, and flexibility of distributed applications.

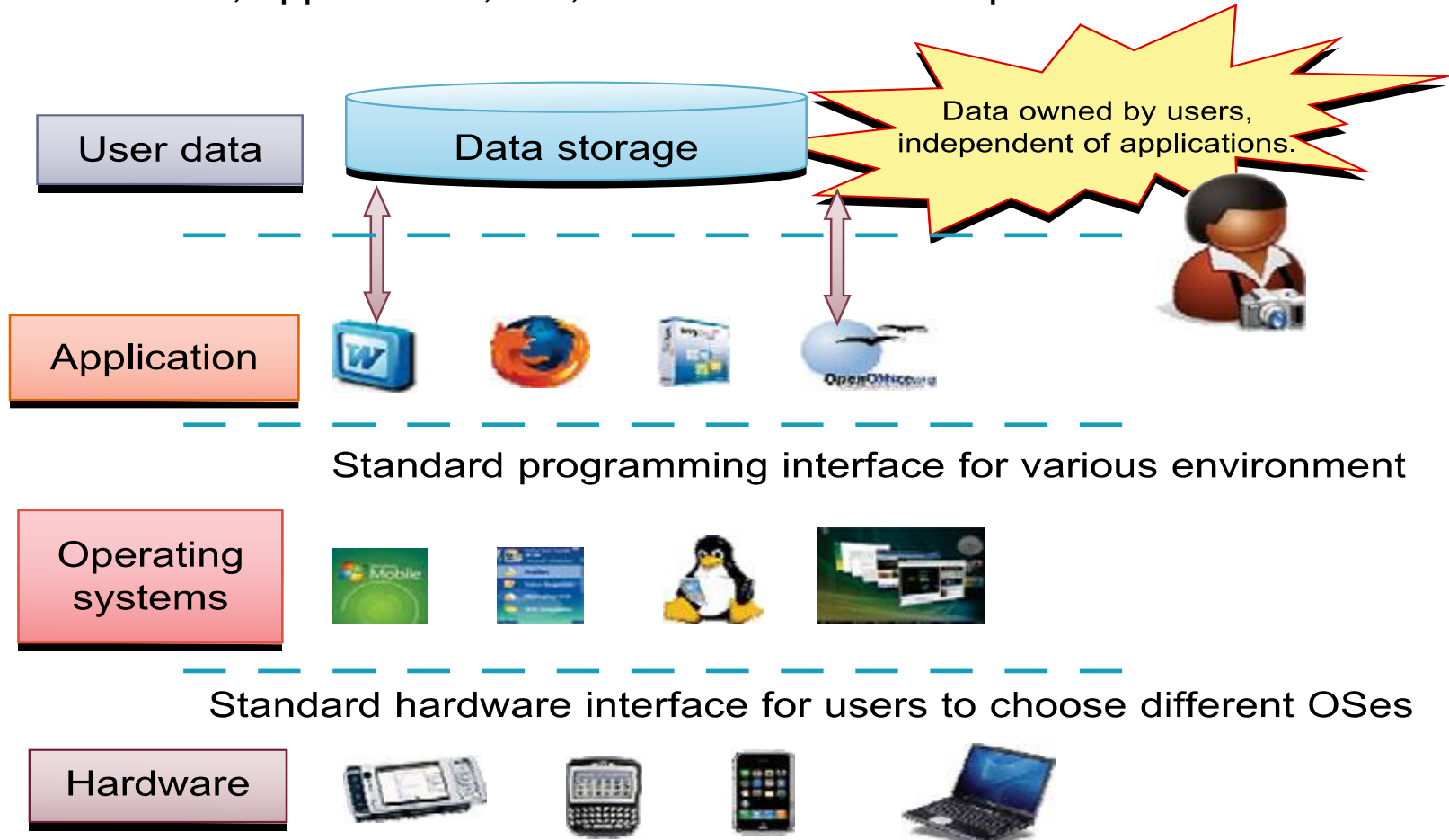
Distributed Operating Systems

Three approaches for distributing resource management functions in a distributed computer system (See 2nd row in the table).

Distributed OS Functionality	AMOEBA Developed at Vrije University [46]	DCE as OSF/1 by Open Software Foundation [7]	MOSIX for Linux Clusters at Hebrew University [3]
History and Current System Status	Written in C and tested in the European community; version 5.2 released in 1995	Built as a user extension on top of UNIX, VMS, Windows, OS/2, etc.	Developed since 1977, now called MOSIX2 used in HPC Linux and GPU clusters
Distributed OS Architecture	Microkernel-based and location-transparent, uses many servers to handle files, directory, replication, run, boot, and TCP/IP services	Middleware OS providing a platform for running distributed applications; The system supports RPC, security, and threads	A distributed OS with resource discovery, process migration, runtime support, load balancing, flood control, configuration, etc.
OS Kernel, Middleware, and Virtualization Support	A special microkernel that handles low-level process, memory, I/O, and communication functions	DCE packages handle file,time, directory, security services, RPC, and authentication at middleware or user space	MOSIX2 runs with Linux 2.6; extensions for use in multiple clusters and clouds with provisioned VMs
Communication Mechanisms	Uses a network-layer FLIP protocol and RPC to implement point-to-point and group communication	RPC supports authenticated communication and other security services in user programs	Using PVM, MPI in collective communications, priority process control, and queuing services

Transparent Computing Environment

- Transparent computing infrastructure for future computing platforms.
- The user data, applications, OS, and hardware are separated into four levels.



Separates user data, application, OS, and space – good for cloud computing.

Transparent Computing Environment

- The OS provides clear interfaces, standard programming interfaces, or system calls to application programmers.
- In cloud infrastructure, the hardware will be separated by standard interfaces from the OS.
 - Users will be able to choose from different OSes on top of the hardware devices they prefer to use.
- To separate user data from specific application programs, users can enable cloud applications as SaaS.
 - Users can switch among different services. The data will not be bound to specific applications.

Parallel and Distributed Programming Models

- Programming models for distributed computing with expected scalable performance and application flexibility.
- Google's MapReduce and BigTable are for effective use of resources from Internet clouds and data centers.
- Service clouds demand extending Hadoop, EC2, and S3 to facilitate distributed computing over distributed storage systems.

Parallel and Distributed Programming

Table 1.7 Parallel and Distributed Programming Models and Tool Sets

Model	Description	Features
MPI	A library of subprograms that can be called from C or FORTRAN to write parallel programs running on distributed computer systems [6,28,42]	Specify synchronous or asynchronous point-to-point and collective communication commands and I/O operations in user programs for message-passing execution
MapReduce	A Web programming model for scalable data processing on large clusters over large data sets, or in Web search operations [16]	<i>Map</i> function generates a set of intermediate key/value pairs; <i>Reduce</i> function merges all intermediate values with the same key
Hadoop	A software library to write and run large user applications on vast data sets in business applications (http://hadoop.apache.org/core)	A scalable, economical, efficient, and reliable tool for providing users with easy access of commercial clusters

Performance, Security, and Energy Efficiency

- Fundamental design principles along with rules for building massively distributed computing systems.
 - Such clusters, grids, P2P networks, and Internet clouds.
- These principles and issues include:
 - Scalability,
 - Availability,
 - Programming Models, And
 - Security

Performance Metrics

In a distributed system, performance is attributed to a large number of factors.

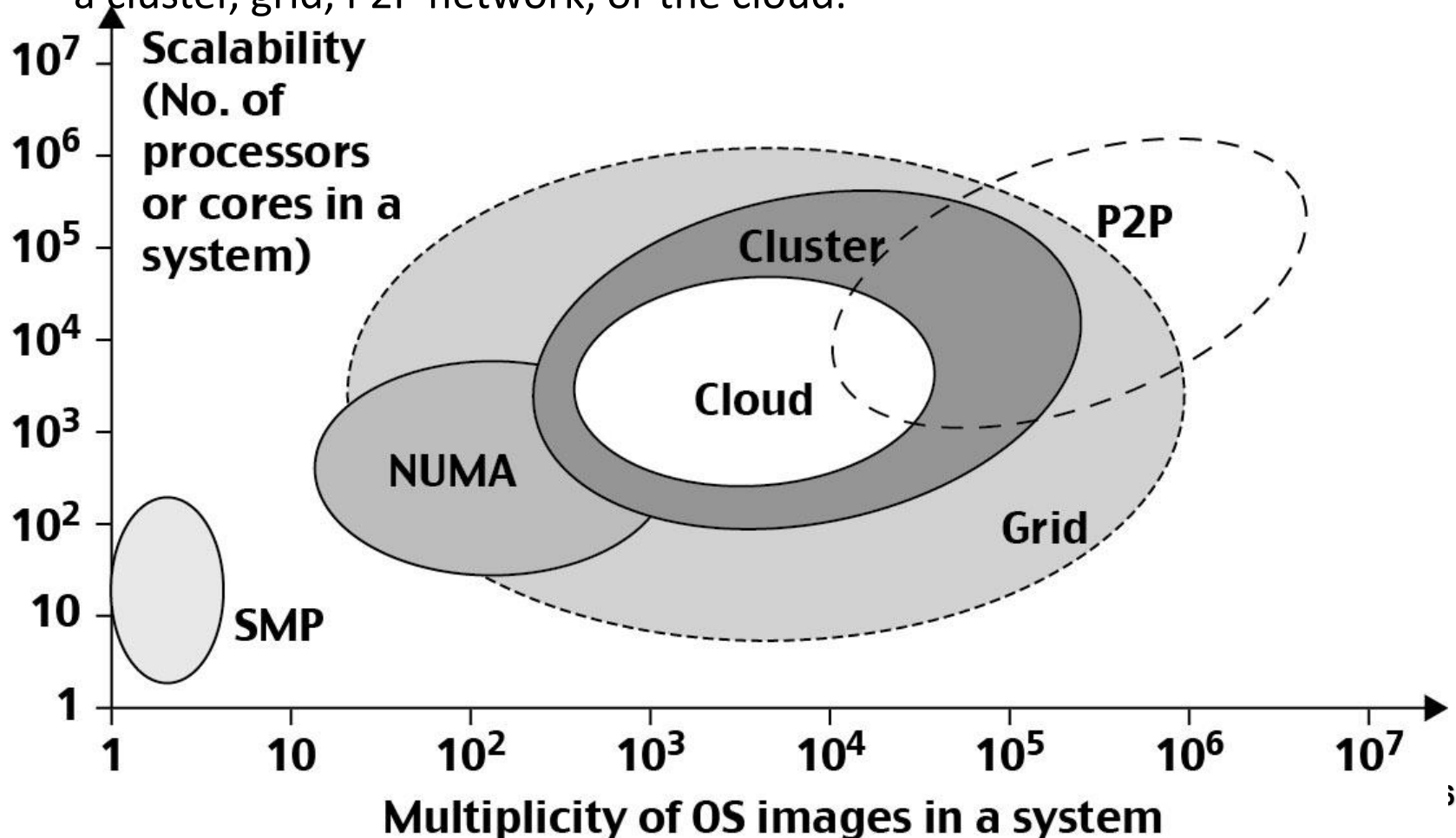
- CPU speed in MIPS
- Network bandwidth in Mbps
- System throughput is often measured in MIPS, Tflops (tera floating-point operations per second), or TPS (transactions per second).
- Other measures include **job response time** and **network latency**.
 - An interconnection network that has low latency and high bandwidth is preferred.
- System overhead is often attributed to OS boot time, compile time, I/O data rate, and the runtime support system used.
- Other performance-related metrics include:
 - QoS for Internet and web services,
 - system availability and dependability, and
 - security resilience for system defense against network attacks.

Dimensions of Scalability

- Size – increasing performance by increasing machine size
- Software – upgrade to OS, libraries, new apps.
- Application – matching problem size with machine size
- Technology – adapting system to new technologies

System Scalability vs. OS Multiplicity

- Scalable performance implies that the system can achieve higher speed by adding more processors or servers, enlarging the physical node's memory size, extending the disk capacity, or adding more I/O channels.
- The OS image is counted by the number of independent OS images observed in a cluster, grid, P2P network, or the cloud.



Amdahl's Law

- Consider the execution of a given program on a uniprocessor workstation with a total execution time of T minutes.
- Now, let's say the program has been partitioned for parallel execution on a cluster of many processing nodes.
- Assume that a fraction α of the code must be executed sequentially, called the sequential bottleneck.
- Therefore, $(1 - \alpha)$ of the code can be compiled for parallel execution by n processors.
- The total execution time of the program is calculated by

$$\alpha T + (1 - \alpha)T/n$$

Where:

- the first term is the sequential execution time on a single processor.
- the second term is the parallel execution time on n processing nodes.

Amdahl's Law

- All system or communication overhead is ignored here. The I/O time or exception handling time is also not included in the following speedup analysis.
- Amdahl's Law states that the **speedup** factor of using the n-processor system over the use of a single processor is expressed by:

$$\text{Speedup} = S = T / [\alpha T + (1 - \alpha)T/n] = 1 / [\alpha + (1 - \alpha)/n]$$

- The maximum speedup of n is achieved only if the sequential bottleneck α is reduced to zero or the code is fully parallelizable with $\alpha = 0$.
 - For example, if $\alpha = 0.25$ or $1 - \alpha = 0.75$, the maximum speedup achieved is 4, even if we use hundreds of processors.
- Amdahl's law teaches us that we should make the sequential bottleneck as small as possible. Increasing the cluster size alone may not result in a good speedup in this case.

Efficiency

- Efficiency is a measure of the fraction of time for which a processing element is usefully employed
- Mathematically, it is given by

$$E = S/n$$

Where: S is the speedup

n is the number of processing elements (processors or computers)

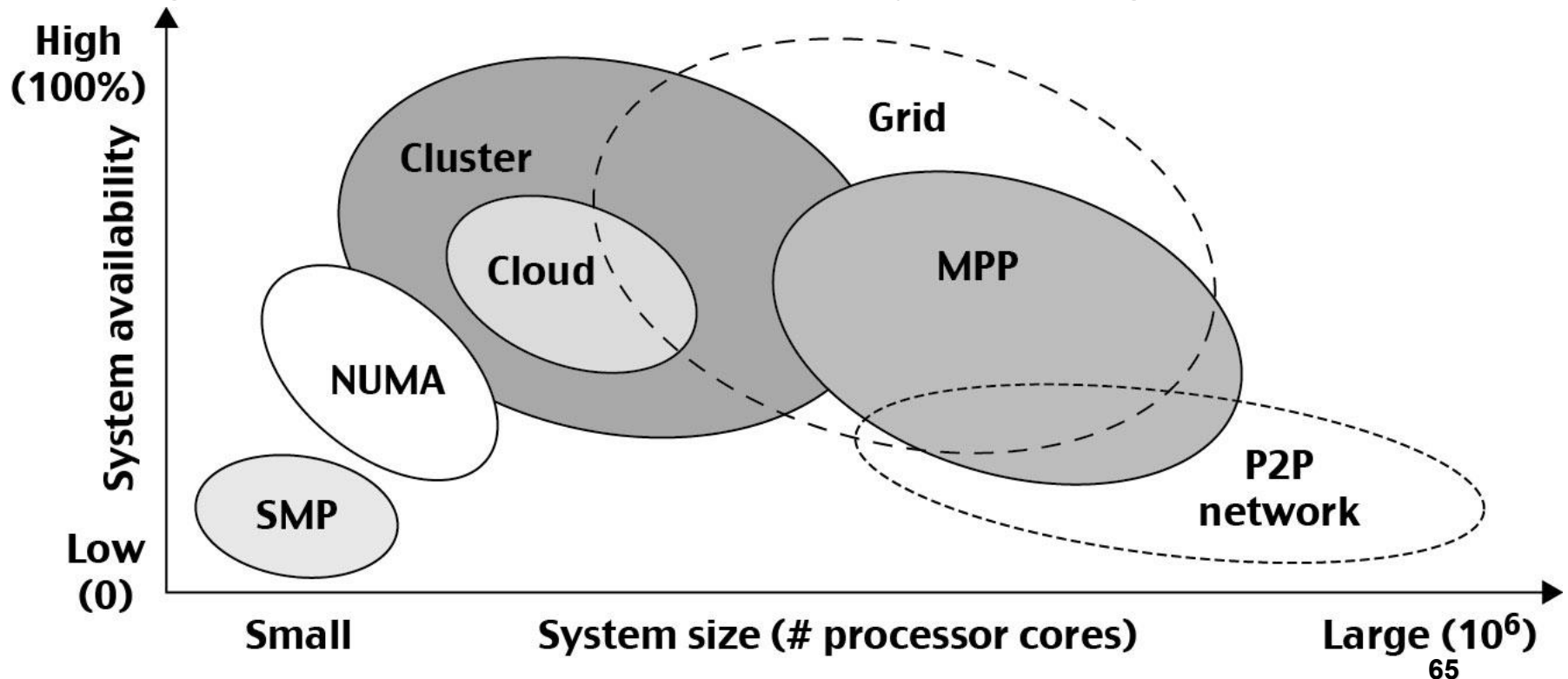
- Following the bounds on speedup, efficiency can be as low as 0 and as high as 1.

System Availability vs. Configuration Size :

- A system is highly available if it has a long mean time to failure (MTTF) and a short mean time to repair (MTTR). System availability is formally defined as follows:

$$\text{System Availability} = \text{MTTF} / (\text{MTTF} + \text{MTTR})$$

- As a distributed system increases in size, availability decreases due to a higher chance of failure and a difficulty in isolating the failures.

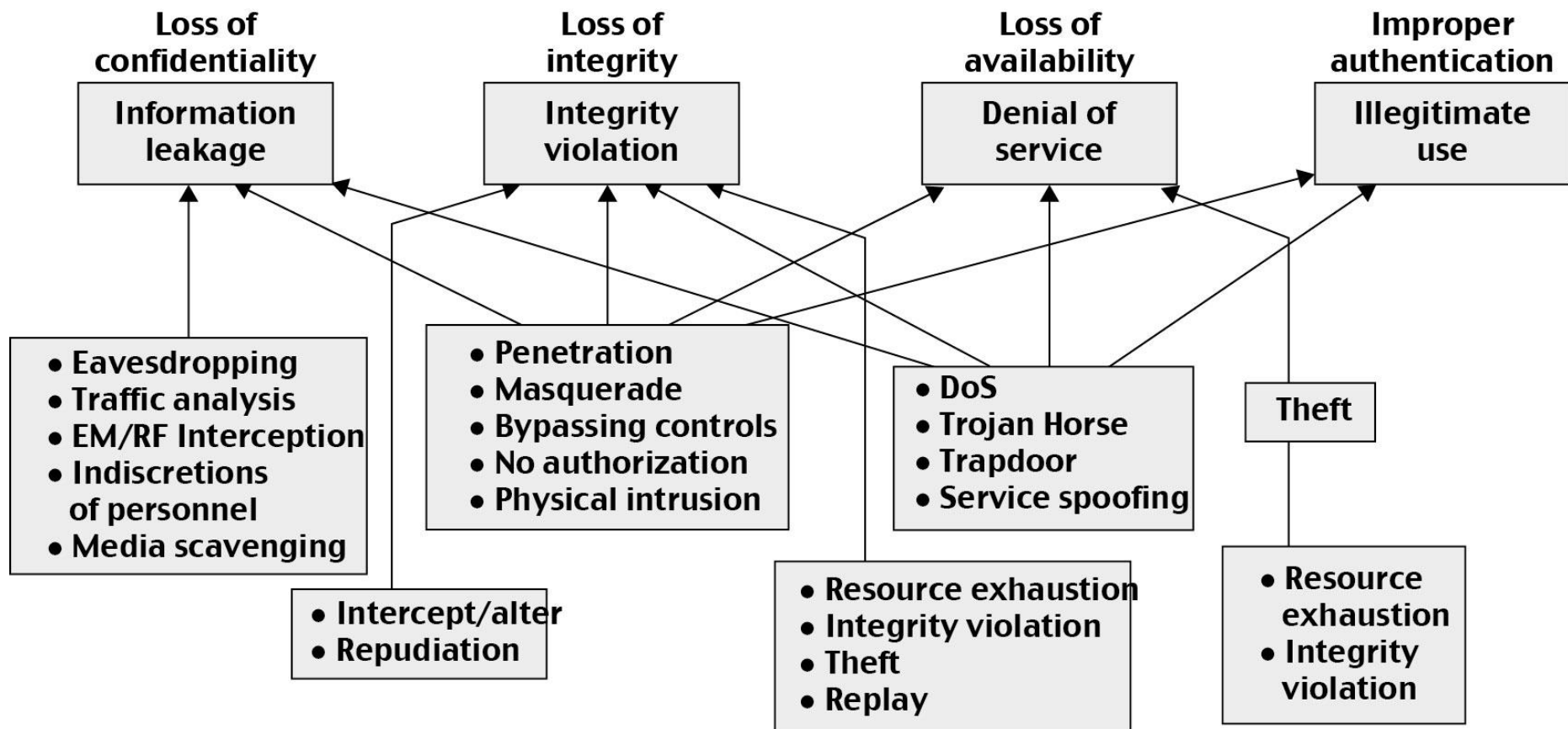


System Availability vs. Configuration Size

- Availability factors include hardware, software, and network components.
- Any failure that will pull down the operation of the entire system is called a **single point of failure**.
- Design a dependable computing system with no single point of failure.
- Adding hardware redundancy, increasing component reliability, and designing for testability will help to enhance system availability and dependability.
- The effects on system availability are estimated by scaling the system size in terms of the number of processor cores in the system.

Security: System Attacks and Network Threats

Various system attacks and network threats to the distributed and cloud systems, resulting in 4 types of losses.



Security: System Attacks and Network Threads

Security Responsibilities in the cloud

- The SaaS model relies on the cloud provider to perform all security functions.
- The IaaS model wants the users to assume almost all security functions, but to leave availability in the hands of the providers.
- The PaaS model relies on the provider to maintain data integrity and availability, but burdens the user with confidentiality and privacy control.

Energy Efficiency in Distributed and Cloud Computing

Performance goals

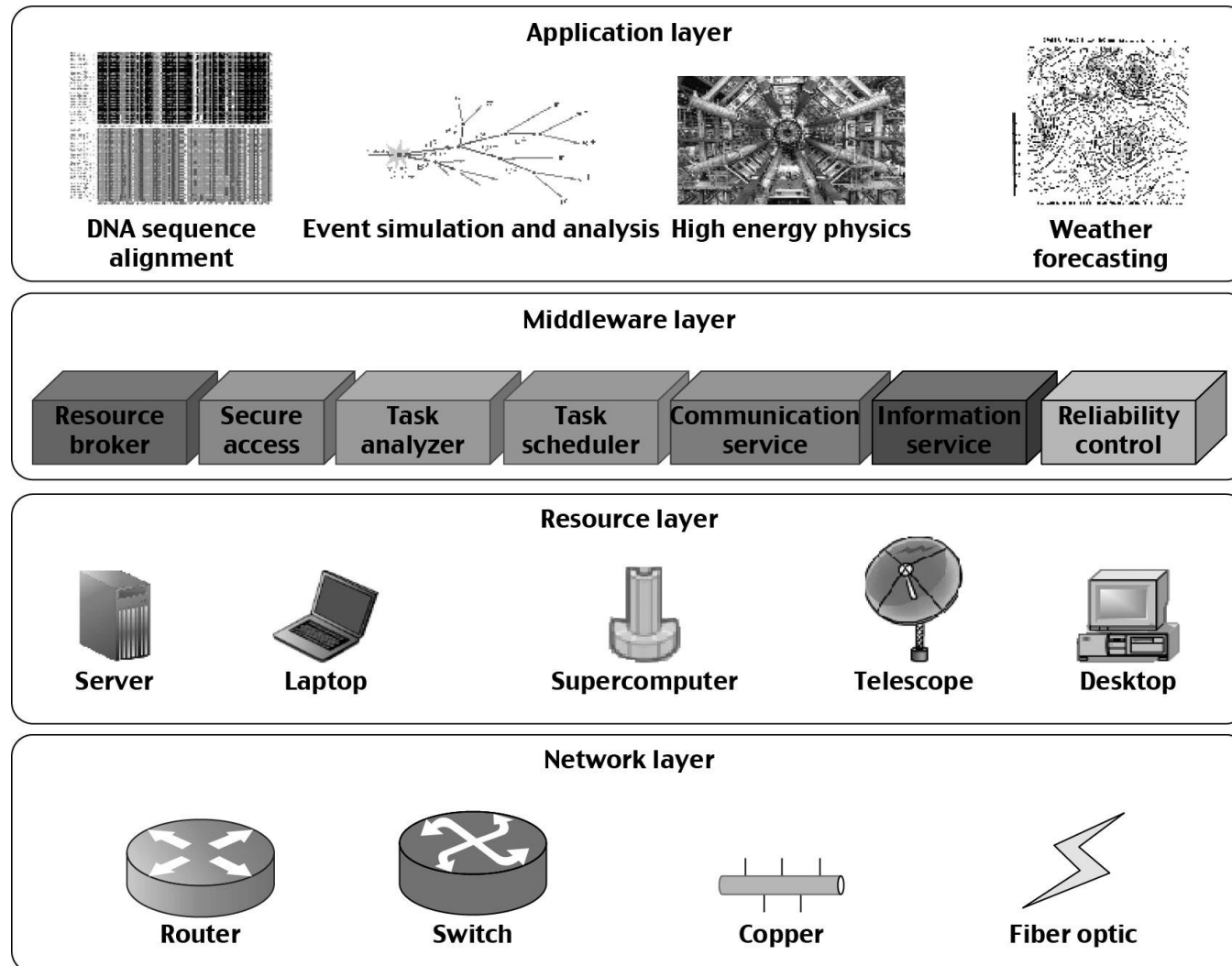
- High performance,
- High throughput,
- Energy Efficiency
 - Power cost, cooling cost
- Workload and resource outsourcing

Energy Consumption of Unused Servers

- It was estimated in the past that, on average, one-sixth (15 percent) of the full-time servers in a company are left powered on without being actively used (i.e., they are idling) on a daily basis.
- In the cloud this is crucial and has changed a lot.
 - Server consolidation and server containment

Energy Efficiency in Distributed and Cloud Computing

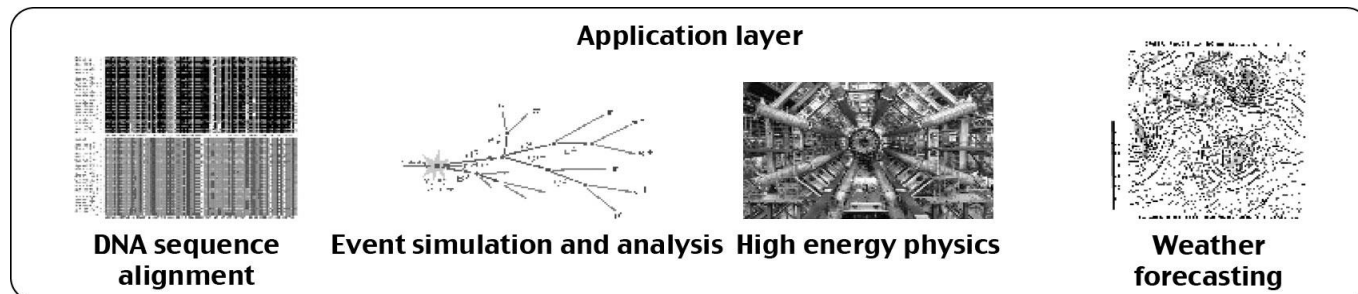
Power management issues in distributed and cloud computing platforms can be categorized into four layers



Energy Efficiency in Distributed and Cloud Computing

Application Layer

- Develop energy-aware applications without affecting performance.
- To do this, explore a relationship between performance and energy consumption.
- Application's energy consumption depends strongly on the number of instructions needed to execute the application and the number of transactions with the storage unit (or memory).
- These two factors (compute and storage) are correlated and they affect completion time.



Energy Efficiency in Distributed and Cloud Computing

Middleware Layer

- This layer provides resource broker, communication service, task analyzer, task scheduler, security access, reliability control, and information service capabilities.
- It is also responsible for applying energy-efficient techniques, particularly in task scheduling.
 - Scheduling aims at minimizing **makespan**, that is, the execution time of a set of tasks.
- Distributed computing systems necessitate a new cost function covering both **makespan** and energy

consumption.

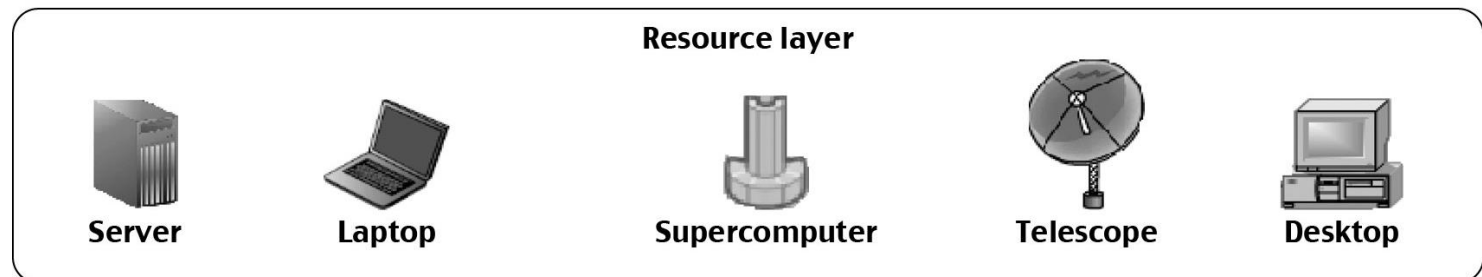
Middleware layer



Energy Efficiency in Distributed and Cloud Computing

Resource Layer

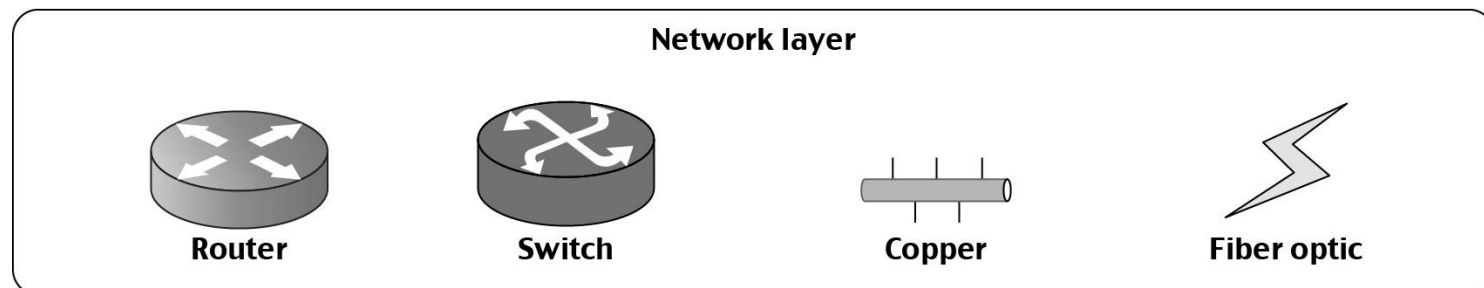
- Efficient power management of hardware (particularly for processors) and operating systems.
- Dynamic power management (DPM) and dynamic voltage-frequency scaling (DVFS) are two popular methods incorporated into recent computer hardware systems .
 - In DPM, hardware devices, such as the CPU, have the capability to switch from idle mode to one or more lower- power modes.



Energy Efficiency in Distributed and Cloud Computing

Network Layer

- How to measure, predict, and create a balance between energy consumption and performance.
- Two major challenges to designing energy-efficient networks are:
 - The models should represent the networks comprehensively as they should give a full understanding of interactions among time, space, and energy.
 - New, energy-efficient routing algorithms need to be developed. New, energy-efficient protocols should be developed against network attacks.



Modern Distributed and Cloud Computing

- Adopt new technologies in current-generation datacenter designs.
- This leads to low construction and operational costs, simple resource management, high usability, high security and reliability, and low energy consumption.

References

1. K. Hwang, G. Fox, and J. Dongarra, *Distributed and Cloud Computing: from Parallel Processing to the Internet of Things* Morgan Kauffmann Publishers, 2011
2. R. Buyya, J. Broberg, and A. Goscinski (eds), *Cloud Computing: Principles and Paradigms*, ISBN-13: 978-0470887998, Wiley Press, USA, February 2011.
3. T. Chou, *Introduction to Cloud Computing: Business and Technology*, Lecture Notes at Stanford University and at Tsinghua University, Active Book Press, 2010.
4. T. Hey, Tansley and Tolle (Editors), *The Fourth Paradigm : Data-Intensive Scientific Discovery*, Microsoft Research, 2009.